00　ECS × EFS × Log4j2「Stale file handle」完全解説 v2（6 タスク・共有トレースログ）

6 タスクが同じ tracelog に書く構成で、なぜ Stale になるのか・なぜ 1 タスクだけ正常になるのか・なぜヘルスチェック（Spring MVC）は 200 を返すのか・何から疑い、どう確かめ、どう直すか
2026年10月10日時点（Log4j 2.25.3 のソースコード・AWS 公式ドキュメント・Linux／NFS の一次情報・Docker での 6 タスク再現実験で確認）

■ ご質問への答え（要約）

質問	答え
Q1　何が起きているのか（一言で）	6 タスクの Log4j2 が、EFS 上の同じ tracelog を「自分だけが使っている」と思い込み、毎時 00 分すぎにそれぞれロールオーバ（今のファイルを trace.log.<日時> に改名し、新しい tracelog を作る）をする。2 番目以降のタスクは、前のタスクが作ったばかりの tracelog を「空なら削除」し、中身があれば同じ名前の保存ファイルを「上書き」して消す。消された実体を開いたまま書いていた別のタスクが Stale file handle（古くなった引換券）になる。Java の例外は Linux のエラー番号 ESTALE（116）をそのまま文字にしたもの。
Q2　仮説 1「起動中に、テストで EFS のファイルシステムを作り直した」は成り立つか	現象としては起こりうる（AWS 公式のトラブルシュートにも「消した FS と同じ IP で作り直すと bad file handle になる」とある）。ただしそのときは /mnt/logs 全体が壊れ、tracelog と同じ親ディレクトリの下（mid/{タスクID}）にある JBoss の server.log・GC ログ・アクセスログも同時に書けなくなり、全タスクがそろって壊れ、タスクを入れ替えるまで直らない（多くは「エラー」より「応答が止まる」になる）。今回は JBoss のログが書け続け、tracelog だけが壊れているので、今回の主因ではない（可能性は低い）。再現実験 C4 でも、作り直すと全部が壊れた。
Q3　仮説 2「全タスクが同じディレクトリの同じ tracelog に書いている」は成り立つか	成り立つ。最有力（ほぼ確定）。「tracelog だけ壊れる」「タスクごとのディレクトリのログは無事」「一部のタスクだけエラー」「正常なタスクが時間とともに入れ替わる」をすべて説明できる。Log4j 2.25.3 のソースで、ロールオーバが同じ名前の保存ファイルを上書きすること（Files.move の ATOMIC_MOVE＋REPLACE_EXISTING）、空のファイルなら消すこと、書き込みに失敗しても開き直さないことを確認した。6 タスクの再現実験でも同じ例外が出た（15_検証結果）。
Q4　6 タスクのうち 1 タスクだけ正常、はありうるか	ありうる。むしろ毎時起きる「ふつうの姿」。00 分すぎに最後にロールオーバしたタスクだけが tracelog に正しく書ける。ほかの 5 タスクは、次の 00 分まで書くたびに Stale（交換の間があいたときは、そのうち 1 タスクがエラーを出さずに前の時間帯の保存ファイルへ書く）。どのタスクが「正常」かは毎時入れ替わる。再現実験では 12 回の境目のうち 12 回で、正しく書けたのは最後にロールオーバした 1 タスクだけだった（06_1タスクだけ正常・15_検証結果）。
Q5　今すぐやることは	① ECS Exec で 6 タスクに tools/tracelog-fd-check.sh を流し「CURRENT 1・残りは STALE（ARCHIVE 0〜1）」の形かを見る（毎時 02 分以降に）　② EFS 上の trace.log.* の行数と中身を見る（1 タスク分しか無ければ確定。名前の時間と中身がずれていることもある）　③ tracelog をタスクごとのディレクトリ（またはタスクごとのファイル名）に分けてデプロイする。再起動だけでは次の 00 分にまた壊れる（10_緊急対応）。
Q6　根本対策は	「1 つのファイルに書くのは 1 タスク（1 つの JVM）だけ」にする。JBoss のログと同じく、-Diwin.log を mid/{タスクID} に向ける（Log4j2 の設定は変えずに済む。再現実験 C3 で Stale 0・欠け 0 を確認）。あわせてレイアウトに時刻とタスク名を入れる。できればトレースログの正本を CloudWatch Logs（サイドカー／FireLens／awslogs）に置く（11_恒久対策）。

Q7 運用で避けるには	動いているタスクが使う EFS 上のファイル・ディレクトリを「消さない・動かさない・作り直さない」。EFS の作り直しは、サービスを 0 タスクにしてから行う。共有ファイルのままの間は、デプロイを毎時 00 分の前後に重ねない（12_運用回避）。
Q8 Spring MVC のヘルスチェック API が 503 ではなく 200 を返すのは、意図的に catch して投げ直さないといけないからか、Controller の握りつぶし（実装ミス）か	どちらでもない。Stale file handle の例外は Log4j2 のアペンダの中で起き、A4 は ignoreExceptions=true（既定）なので、Log4j2 自身が受け止めて標準エラーに出すだけで終わる（AppenderControl）。さらに log4j-api も、ログ呼び出しの中の例外を呼び出し元へ投げない設計（LOG4J2-1990）。そのため logger.debug() を呼んだ Controller には何も届かず、catch して投げ直すこともできない。ヘルスチェック API がトレースログの状態を調べていないので 200 になる、というのが正しい見方（設計上の不足であって、握りつぶしの実装ミスではない）。なお Spring MVC では、処理されない例外はふつう 500 で、503 にするには明示的な対応付けが要る（13_ヘルスチェック）。
Q9 ヘルスチェック（ELB から 30 秒ごと・常に 200）はどうするか	ヘルスチェックを「ログが書けるか」で失敗させるのは、今の構成では危険（毎時 4〜5 タスクが不健康になり、ECS が一斉に入れ替え、新しいタスクも次の 00 分にまた壊れる）。先に根本対策をし、ログの失敗は CloudWatch のアラームで知らせる。ELB のヘルスチェックは「リクエストを処理できるか」に絞り、トレースログの状態は専用の API（Spring MVC の例を 13 に記載）で見る。

■ 目次（シート名をクリックすると移動）

シート	内容
00_表紙	要約・目次・凡例
01_結論	2 つの仮説の判定・決め手の観察・原因の順位と対処の早見表
02_たとえ話	小学生にもわかる「6 人で 1 冊のノート」のお話
03_歴史と背景	NFS・ファイルハンドル・ESTALE・ロールオーバ・EFS・コンテナのログが生まれた順番と、必要になった経緯
04_動作原理	名前・実体・FD／NFS と EFS の約束／Log4j2 2.25.3 のロールオーバの中身／例外の出方
05_動作イメージ	13:00 の連鎖を 1 コマずつ（6 タスク）・例外が出るまで・仮説 1 のとき
06_1タスクだけ正常	なぜ 1 タスクだけ正常になるか・「正常」の 3 つの意味・例外のパターン
07_原因一覧	可能性の高い順に全 10 項目。仕組み・見分け方・緊急／恒久／運用の対処
08_確認方法	事象の確かめ方（ログ・EFS 上のファイル・タスクの中・AWS の記録）
09_切り分け	はい／いいえで原因を絞る手順と、観察と原因の対応表
10_緊急対応	今すぐ止める手順（再起動では直らない理由つき）
11_恒久対策	1 ファイル 1 書き手・正本を CloudWatch Logs・設定例とスクリプト
12_運用回避	やってはいけない操作・作り直しの手順・チェックリスト
13_ヘルスチェック	Spring MVC の API が 200 を返す理由（握りつぶしか設計か）・503 の返し方・30 秒ごとのヘルスチェックの安全な設計
14_コマンド集	調査・確認に使うコマンドを 1 か所に（bash／PowerShell／Logs Insights）
15_検証結果	Docker で 6 タスク＋NFS を立てて再現した結果（Log4j 2.25.3）
16_用語集	用語を小学生向けのたとえつきで

17_参考資料	根拠にした公式ドキュメント・ソースコード・一次情報

■ 凡例

表記	意味
★★★★★ ～ ☆☆☆☆☆	この環境（ECS Fargate＋EFS の共有パス＋Log4j2）で原因である見込み（筆者の評価）。★ が多いほど先に疑う
ESTALE / Stale file handle	Linux のエラー番号 116。「その引換券（ファイルハンドル）の品物は、もう無い」
実体（inode）・#0 #1 …	ファイルの中身そのもの。名前（tracelog）とは別物。図では #番号 で表す
T1～T6	6 つの ECS タスク（それぞれが別の NFS クライアント）。図の番号は「その時間にロールオーバした順」
CURRENT / ARCHIVE / STALE	診断ツールの判定。正しく tracelog に書けている／前の時間帯の保存ファイルに書いている／Stale
trace.log.…-12	trace.log.2026-10-10-12 のような、12 時台として保存されたファイルの略
濃い灰色のセル	失敗・エラー・ログが失われる場面。薄い灰色は要所

作成: 2026-10-10 ／ フォント: Meiryo UI ／ 配色: モノトーン ／ 本文は Markdown 版と同じ

01　結論：原因は「6 タスクが同じ tracelog をそれぞれロールオーバしていること」

仮説 2（全タスクが同じ tracelog に書く）が最有力で、ほぼ確定。仮説 1（EFS のファイルシステムの作り直し）は、起きればマウント全体が壊れるはずで、同じディレクトリの下（mid/{タスクID}）にある JBoss のログが書けている今回の様子と合わない。6 タスクのうち 1 タスクだけが正常、という状態は毎時ふつうに起きる。ヘルスチェック API が 200 を返すのは Controller の握りつぶしではなく、Log4j2 が例外をアプリに渡さない設計（ignoreExceptions=true が既定）のため。

■ ひとことで　Log4j2 の RollingFile は「このファイルを使うのは自分（この JVM）だけ」という前提で作られている。6 つのタスク（6 つの JVM、6 つの NFS クライアント）が同じ tracelog を使うと、毎時 00 分すぎに 6 回ロールオーバが起き、後からロールオーバしたタスクが、先のタスクのファイルを消す（空なら削除・中身があれば同じ名前の保存ファイルを上書き）。消されたファイルを開いたまま書いていたタスクに、EFS が「その実体はもう無い（STALE）」と返す。その例外は Log4j2 の中で処理され、アプリ（Spring MVC）には届かない。

■ 2 つの仮説の判定

仮説	判定	理由	確かめ方
仮説 2：アプリのトレースログは、すべてのタスクが同じディレクトリの同じファイル（/mnt/logs/front/logs/inter-api/tracelog）に書いている	成り立つ（最有力・ほぼ確定）	Log4j2 のロールオーバは JVM ごとに独立して動く。2 番目以降のタスクは、前のタスクが作ったばかりの tracelog を「空なら削除」し、中身があれば同じ名前の保存ファイル（trace.log.<日時>）を上書きして改名する（2.25.3 のソースで確認）。どちらでも誰かの実体が消え、EFS は別のクライアントに消されたファイルへの操作に STALE を返す。タスクごとに分けた JBoss のログが無事なこととも合う。6 タスクの再現実験で同じ例外と「正しく書けるのは毎回 1 タスク」を確認	08_確認方法 の C-1〜C-4（各タスクの FD の判定、trace.log.* の更新時刻と中身、エラーの時刻が毎時 00 分すぎか）
仮説 1：アプリを起動している最中に、テスト用途で EFS のファイルシステムを作り直した	今回の主因としては成り立たない（可能性は低い）。ただし「ディレクトリやファイルの作り直し」なら引き金にはなりうる	FS を作り直すと、古い FS の引換券はすべて無効になる。tracelog だけでなく /mnt/logs の下の全ファイル（同じ inter-api の下の mid/{タスクID} の server.log・GC・アクセスログも）が全タスクで同時に壊れ、タスクを入れ替えるまで直らない（多くは応答が止まる）。今回の「tracelog だけ」「一部のタスクだけ」と合わない（再現実験 C4 でも全部壊れた）	08_確認方法 の C-5〜C-7（ls /mnt/logs が動くか、JBoss のログも失敗しているか、CloudTrail の DeleteFileSystem／CreateFileSystem の時刻）

■ 決め手になる観察（どちらの仮説で説明できるか）

観察	仮説 2（共有ファイル）	仮説 1（FS の作り直し）
壊れているのはアプリのトレースログ（共有の tracelog）だけ	○ 共有しているファイルだけが壊れる	× 同じマウントの全ファイルが壊れるはず
JBoss の server.log・GC ログ・アクセスログ（tracelog と同じ inter-api の下の mid/{タスクID}）は書け続け、そこにトレースログのエラーまで書けている	○ 1 ファイル 1 書き手なので誰にも消されない	× 同じディレクトリの下なので一緒に壊れる
6 タスクのうち一部だけがエラーになる（1 タスクは正常なことがある）	○ 最後にロールオーバしたタスクだけが正しく書ける	× 全タスクが同時に壊れる
エラーが毎時 00 分すぎから出始める／正常なタスクが入れ替わる（要確認）	○ ロールオーバの時刻と順番で決まる	× 時刻と関係なく、作り直した瞬間から
再起動（タスクの入れ替え）でしばらく直るが、また出る	○ 次の 00 分でまた連鎖する	△ 入れ替えれば直り、二度と出ない
ls /mnt/logs や JBoss のログの書き込みは正常	○	×（ls も Stale・bad file handle か、応答なし）
ヘルスチェック API は 200 を返し続ける	○ Log4j2 が例外をアプリに渡さないので、どの Controller にも届かない	△ ヘルスチェックがファイルに触れなければ 200、触れれば応答が止まる

■ 新しい問い：Spring MVC のヘルスチェック API が 503 ではなく 200 を返すのはなぜか

問い	答え
Stale file handle の例外は、Java 側で意図的に catch してスローし直さないと、Controller のヘルスチェック API で検知できないのか	そもそも Controller まで届かない。例外は Log4j2 のアペンダの中（OutputStreamManager）で起き、ignoreExceptions=true（既定）の A4 では AppenderControl が処理して終わる（StatusLogger で標準エラーに出すだけ）。logger.debug() を呼んだ側には何も投げられないので、業務の Controller でもヘルスチェックの Controller でも catch のしようがない（2.25.3 のソースと実験で確認）
それとも単なる実装ミス（Controller 側で握りつぶしている）と推測するのが正しいか	今回の 200 は握りつぶしではなく、設計どおりの動き。ヘルスチェックがトレースログの状態を何も調べていないので 200 になる（調べる仕組みが無い＝設計上の不足）。ただし、Controller に catch(Exception) で 200 を返す処理や、@ControllerAdvice で例外を 200 に変える処理があれば、それは別の問題（握りつぶし）なので、コードで確認する（13 の 13-4）
例外が出れば 503 になるのか	ならない。Spring MVC では、処理されない例外はふつう 500（サーブレットコンテナのエラー）になる。503 にするには ResponseEntity.status(503)・ResponseStatusException・@ResponseStatus・@ExceptionHandler などで明示的に決める必要がある
では 503 を返すべきか	今の構成のままではやめる。毎時 4〜5 タスクが不健康になり、ECS が一斉に入れ替え、新しいタスクも次の 00 分にまた壊れる。先に 1 ファイル 1 書き手に直し、ログの失敗はアラームで知らせる。どうしても API で見たいなら、トレースログの状態を自分で調べる専用の API を作る（13 に例）

■ 原因の順位（早見表。詳しくは 07_原因一覧）

順位	原因	可能性	ひとこと
1	6 タスクが同じ tracelog に書き、各 JVM の Log4j2 が毎時ロールオーバ（空なら削除・中身があれば上書きで改名）	★★★★★	今回の観察をすべて説明できる
2	デプロイ中の新旧タスクの重なり（最大 12 タスクが同じファイル）・途中で起動したタスクの即時ロールオーバ	★★★★☆	1 を強める。デプロイ直後に増える
3	ほかの書き手（バックコンテナの設定ミス・同じ JBoss の別の WAR・別環境・別サービス）が同じパスに書いている	★★★☆☆	書き手が増えるほど悪化
4	ログの片付け（find -delete・Lambda・cron・手作業 rm）が動いているタスクのファイルを消した	★★★☆☆	片付けがあれば必ず確認
5	EFS 上のディレクトリを消して作り直した（テストの初期化・デプロイスクリプト・手作業）	★★☆☆☆	仮説 1 の「読み替え版」。一時的な引き金
6	EFS のファイルシステム（やマウントターゲット）を、タスクが動いたまま作り直した	★☆☆☆☆	仮説 1 そのもの。マウント全体が壊れる
7	アクセスポイント／rootDirectory の削除・作り直し	★☆☆☆☆	根元が Stale になる
8	AWS DataSync・復元・同期ツールが同じパスのファイルを置き換えた	★☆☆☆☆	置き換え＝消して作り直し
9	ネットワーク切断・NFS のリース切れ・クライアントの不具合	☆☆☆☆☆	ふつうは EIO・応答なしで、Stale ではない
10	EFS 側の障害・メンテナンス	☆☆☆☆☆	引換券は保たれる設計。AWS Health で確認

■ 対処の早見表

区分	やること	効果	注意
確認（今日）	6 タスク全部で tools/tracelog-fd-check.sh を実行し、CURRENT・ARCHIVE・STALE の数を見る。EFS 上の trace.log.* の更新時刻と中身を見る	仮説 2 を数分で確定できる	ECS Exec が有効なタスクだけ。無効なら EFS をマウントした別の場所から見る
緊急（今日〜明日）	tracelog の書き先をタスクごとに分ける（JBoss のログと同じ mid/{タスクID} に -Diwin.log を向ける／fileName に ${hostName} を入れる）設定でデプロイ	次の 00 分から連鎖が止まる	再起動だけでは直らない。デプロイ中は新旧が重なるので 00 分の前後を避ける
恒久	1 ファイル 1 書き手を設計の約束にする。正本を CloudWatch Logs に。レイアウトに時刻・タスク名。片付けは止まったタスクのものだけ	同じ種類の事故が起きなくなる	既存の集計・監査の手順（1 ファイル前提）を見直す
運用	動いているタスクのファイル・ディレクトリを消さない／作り直さない。EFS の作り直しは 0 タスクにしてから	仮説 1 型の事故を防ぐ	テスト環境でも同じ

監視	server.log の「Stale file handle」「Unable to write to stream」「Unable to close stream」を CloudWatch のメトリクスフィルタで数え、アラームにする	気づけるようになる	Log4j2 は同じエラーを間引く（最初の数件のあとは 5 分に 1 回）。件数ではなく「出たか」で判定する
ヘルスチェック	ELB 用は「リクエストを処理できるか」に絞る。トレースログの状態は別の API・アラームで見る（13）	誤った一斉入れ替えを防ぐ	根本対策の前に 503 を返すようにしない

■ 今回の構成の要点（いただいた情報の整理）

項目	値	この件での意味
基盤	ECS on Fargate・6 タスク（各タスクにフロント／バックの 2 コンテナ）	6 タスク＝6 台の別々のマイクロ VM＝EFS から見て 6 つの別々の NFS クライアント
イメージ	UBI 9.8・Red Hat build of OpenJDK 21・JBoss EAP 8.1・Spring MVC	Java の IOException は Linux の ESTALE（116）をそのまま文字にしたもの
ログの置き場所	EFS を /mnt/logs にマウント（フロント・バックとも）	NFSv4.1。複数タスクから同じファイルが見える
リンク（フロント）	/webapp/webapp9mf02/logs → /mnt/logs/front/logs/inter-api	リンクは「開くときの道案内」。開いた後は関係しない（原因ではない）
リンク（バック）	/webapp/webapp9mb02/logs → /mnt/logs/back/logs/inter-api	フロントと別ディレクトリなので、フロントとバックは互いに干渉しない（バックも 6 タスクで同じ tracelog なら、バックでも同じことが起きる）
共有されているファイル	/mnt/logs/front/logs/inter-api/tracelog（6 タスクのフロントが同じファイル）	ここが問題の中心
タスクごとのファイル	JBoss EAP の server.log・GC ログ・アクセスログ：/mnt/logs/front/logs/inter-api/mid/{タスクID}/	1 ファイル 1 書き手なので無事。tracelog と同じ親ディレクトリの下にあるので「ディレクトリや FS は壊れていない」ことの証拠にもなる
Log4j2	log4j-api／log4j-core／log4j-slf4j2-impl 2.25.3	2026年10月時点の最新は 2.25.5（2.25 系）・2.26.1。共有ファイルの問題を直す変更は無い
アペンダ	RollingFile A4：fileName=${sys:iwin.log}/tracelog、filePattern=${sys:iwin.log}/trace.log.%d{yyyy-MM-dd-HH}、TimeBasedTriggeringPolicy	1 時間ごとにロールオーバ。ignoreExceptions は既定の true。レイアウトの指定が無いので既定の %m%n（時刻もタスク名も入らない）
ロガー	jp.iwin.base.tango.ILGLog4j.trace（debug・additivity=false）→ A4	リクエストのたびに書くので、00 分を過ぎると全タスクがほぼ同時にロールオーバする
ヘルスチェック	ELB からフロントへ 30 秒ごと。Spring MVC の Controller で実装した API が常に 200	トレースログの状態は見ていないので、エラー中のタスクにも振り分けられる（13 で理由と設計を説明）

■ 設定の写しで気になった点　いただいた XML の level"="debug" は転記の誤りと思われる（本当にこう書かれていると XML として読めず、A4 自体が作られない）。また、最初の行の「/mnt/logs/front/logsinter-api」は、後の記載（/mnt/logs/front/logs/inter-api）の誤記として読んだ。08 の C-8 の手順で、各コンテナの実際の -Diwin.log の値とリンクの行き先を一度確かめておくと確実。

02　たとえ話：6 人で 1 冊のノートを使うと、なぜ書けなくなるの？

小学生にもわかるように、学校の「きろくノート」のお話にします。むずかしい言葉は、右の（　）にほんとうの名前を書きました。

■ 登場するもの

お話の中	ほんとうの名前	どんなもの？
きろく係の 6 人	6 つの ECS タスク（の中の Java＝JBoss とアプリ）	起きたことをノートに書く人たち。同じ仕事をしている
職員室のたな	EFS（ネットワークの向こうにある、みんなで使えるファイル置き場）	6 人とも同じたなを見ている
きろくノート	tracelog（トレースログのファイル）	今の時間のきろくを書くノート
「12 時のノート」のラベル	trace.log.2026-10-10-12（保存されたファイルの名前）	1 時間ぶん書いたら、ラベルをはってたなにしまう
ノートの番号札	ファイルハンドル（Java では FD）	ノートの『名前』ではなく『番号』で、どのノートか覚えておく札
ノート交換	ロールオーバ	1 時間ごとに、今のノートをしまって新しいノートを出すこと
「そのノートはもうありません」	Stale file handle（ステイル・ファイル・ハンドル）	番号札のノートが、もうたなに無いときに言われること
先生の見回り	ELB のヘルスチェック（30 秒ごと）	「元気？」と聞くだけ。ノートに書けているかは見ない
受付係	ヘルスチェック API（Spring MVC の Controller）	先生に「元気です」（200）と答える係
ノート係の約束「書く人を困らせない」	Log4j2 の ignoreExceptions=true（既定）	ノートに書けなくても、書いた人や受付係には知らせず、職員室のメモ（標準エラー）に小さく書くだけ
自分の名前の引き出し	mid/{タスクID}（JBoss・GC・アクセスログのディレクトリ）	同じたなの、同じ段の中にある、1 人 1 つの引き出し

■ お話 1：1 人で 1 冊なら、何も困らない（JBoss のログ）

JBoss の server.log・GC・アクセスログは、たなの同じ段の中に、1 人 1 つずつ「自分の名前の引き出し」（mid/{タスクID}）があり、そこに自分のノートを入れています。1 時間（や 1 日）たったら、自分のノートにラベルをはってしまい、自分の新しいノートを出します。ほかの人は、その引き出しをさわりません。だから、いつでもちゃんと書けます。今回、JBoss のログが元気に書けているのは、このおかげです。

■ お話 2：6 人で 1 冊のノート、しかも 6 人全員が「ノート交換係」

トレースログは、6 人が 1 冊のノートを使っています。ところが、6 人ともが「1 時間たったら、ノートを交換するのは自分の仕事」と思っています（Log4j2 は、ほかのタスクがいることを知りません）。しかも、ノートには『名前』ではなく『番号札』で書きます。ノートを開いたら、あとは番号札だけを見て書くのです。

▸ **13 時になると…**

1　Aさんが交換する

13 時を過ぎて最初にきろくを書こうとした A さんが、机のノート（12 時台のきろく）に「12 時のノート」のラベルをはってたなにしまい、新しいノート①を机に出します。

▼

2　Bさんも交換する → ここで事件

B さんも「交換は自分の仕事」と思っています。机の上のノート①を見ると、まだ何も書いていない白紙に見えます（A さんが書いた字は、まだ B さんからは見えません）。白紙のノートはしまわずに捨てる決まりなので、B さんは①を捨てて、新しいノート②を出します。A さんの番号札のノート①が、捨てられてしまいました。

▼

3　C･D･E･F さんも交換する

C さんは②を白紙と見て捨てて③を出し、D さんは③を捨てて④を出し…と続きます。交換のたびに、ひとつ前の人のノートが捨てられていきます。

▼

4　もし字が見えていたら

①にもう字が書いてあるのが見えたときは、B さんは①に「12 時のノート」のラベルをはってしまおうとします。でも、たなにはもう「12 時のノート」があるので（同じラベルは 1 冊だけ）、もとの「12 時のノート」を捨ててしまいます。このときは A さんが、「12 時のノート」に 13 時のことを書き続けることになります。どちらにしても、だれかのノートが捨てられます。

▼

5　交換が終わったあと

机の上には F さんのノートだけがあります。A～E さんの番号札のノートは、もう捨てられています。

▼

6　A～E さんが書こうとすると

番号札を見てノートを探しても無いので「そのノートはもうありません」（Stale file handle）と言われます。新しいノートを取りに行くきまりが無いので、14 時の交換まで、書くたびに同じことを言われます。書いたきろくは消えてしまいます。

▸ **その結果**

人	13 時台、どうなる？	ほんとうの判定
F さん（最後に交換した人）	机のノートに正しく書ける。1 人だけ「正常」	CURRENT（正常）

A・B・C・D・E さん	書くたびに「そのノートはもうありません」。書いたことは全部消える	STALE（エラー）
（字が見えていたとき）E さんのような人	書けているが「12 時のノート」に 13 時のことを書いている。エラーは出ない	ARCHIVE（エラーなし・でも前の時間帯のファイル）

■ 大事なところ　だれかがズルをしたわけではありません。6 人とも「自分の決まり」どおりに正しく動いています。問題は「1 冊のノートを 6 人で使い、6 人全員が交換係」という決め方のほうにあります。どの人が F さん（正常）になるかは、毎時「最後に交換した人」なので、毎時入れ替わります。コンピュータで同じことを再現した実験でも、毎回 6 人のうち 1 人だけが正しく書けていました。

■ お話 3：先生の見回り（ヘルスチェック）では気づけない

先生は 30 秒ごとに「元気？」と聞いて回ります。答えるのは各班の受付係（ヘルスチェック API）です。受付係は「元気です」（HTTP 200）と答えます。受付係は、ノートのことを何も知らされていないからです。

ノート係（Log4j2）には「書く人を困らせない」という約束（ignoreExceptions=true）があります。ノートに書けなかったときも、書いた人や受付係には言わずに、職員室のメモ帳（標準エラー → JBoss の server.log の [stderr]）に小さく書いておくだけです。しかも同じことを何度も書くとメモ帳がいっぱいになるので、最初の何回かのあとは 5 分に 1 回しか書きません。

だから受付係は、ノートに書けていないことに気づけません。受付係がわざと隠している（握りつぶしている）のではなく、そもそも知らされていないのです。受付係に気づいてほしいなら、受付係が自分でノートを見に行く（トレースログの状態を調べる）仕組みを作る必要があります。

では「ノートに書けない人は元気じゃないことにする」と決めるとどうなるでしょう。今のやり方のままだと、毎時 4〜5 人が「元気じゃない」になり、そのたびに 4〜5 人が帰らされて、新しい人と入れ替わります。お客さんの用事を受ける人が急に減ってしまいます。だから、先にノートの使い方（1 人 1 冊）を直すのが先です。

■ お話 4：たなごと新しくする（仮説 1：EFS の作り直し）と？

職員室のたなを、まるごと新しいたなに取りかえたとします。すると、6 人全員の番号札が全部使えなくなります。トレースログだけでなく、同じ段の引き出し（mid/{タスクID}）に入っている JBoss の server.log のノートも、GC のノートも、アクセスログのノートも、全部書けなくなります（多くの場合は、たなが見つからずに待たされ続けます）。今回は、同じ段の引き出しの JBoss のノートが元気に書けているので、「たなごと取りかえた」ことが今の原因ではないとわかります。

■ どうなおす？

いま
6 人で 1 冊・6 人とも交換係

→

なおしたあと
1 人 1 冊・交換は自分のノートだけ

JBoss のログと同じやり方。だれも人のノートを捨てない

・1 人に 1 冊のノート（タスクごとのディレクトリ、またはタスクごとのファイル名）にする。これがいちばん大事。

・ノートの 1 行ごとに「時刻」と「だれが書いたか」を書く（レイアウトに時刻とタスク名を入れる）。あとで調べやすくなる。

・使っている人がいるノートやたなを、勝手に捨てたり作り直したりしない（片付けは、帰った人のノートだけ）。

・できれば、ノートの正本は学校の「記録センター」（CloudWatch Logs）に送る。たなが変わっても、きろくは残る。

03　歴史と背景：引換券・ロールオーバ・共有ストレージは、なぜ生まれたのか

今回の部品は、それぞれ別の時代に別の理由で生まれた。組み合わせたときに「1 台のサーバ」の前提が崩れたのが、今回の事故の正体。

■ 年表

年	できごと	なぜ必要だったか・今回との関係
1970 年代	Unix のファイルシステム：名前（ディレクトリの項目）と実体（inode）を分ける	同じ実体に複数の名前をつけ、開いている間は名前を消しても中身を残せる。「開いた後は名前を見ない」はここから
1984〜1985	Sun Microsystems が NFS（Network File System）を開発・公開	ネットワークの向こうのディスクを手元のように使うため
1989	NFSv2 の仕様（RFC 1094）	サーバは誰が何を開いているかを覚えない（ステートレス）。ファイルは名前ではなく「ファイルハンドル」で指す。サーバが再起動しても引換券がそのまま使える
1980〜90 年代	ESTALE（「Stale NFS file handle」）	引換券の品物がサーバ側で消されたことを知らせるエラー番号。今回の例外の正体
1995	NFSv3（RFC 1813）	大きなファイル・非同期の書き込みなど。ステートレスのまま
1996〜2001	log4j が生まれる（EU の SEMPER プロジェクト → IBM チューリッヒ研究所 → 2001 年 1 月に Apache へ）	System.out では足りない「レベル」「出し先」「設定で切り替え」を実現。ファイルへ書く Appender もこのころから
2000 年代前半	log4j 1.2 系の RollingFileAppender／DailyRollingFileAppender が広く使われる	ログが大きくなりすぎる・ディスクがあふれる・日付で探したい、という運用の要望から「ロールオーバ（交換）」が定着
2000〜2003	NFSv4（RFC 3010・3530）	サーバが OPEN やロックを覚える（ステートフル）。インターネット越しにも使える設計に
2002 ごろ	Linux の NFS クライアントに close-to-open（閉じたら書き戻し、開くときに確かめる）一貫性（2.4.20）	複数のクライアントで「順番に」使えば、ほかのクライアントの変更が見える。ただし「同時に追記」は守られない
2004 年 3 月	Spring Framework 1.0	Java EE の重さを避ける軽い枠組み。のちの Spring MVC（Controller・DispatcherServlet）が今回のヘルスチェック API の土台
2006	Red Hat が JBoss を買収	JBoss EAP（企業向けの Java アプリケーションサーバ）へ
2009 年 5 月	AWS Elastic Load Balancing（ELB）	ヘルスチェックに失敗したサーバを振り分けから外す仕組み。「何を確かめれば健康か」は使う側が決める
2010	NFSv4.1（RFC 5661、2020 年に RFC 8881）	セッションなど。「開いている間は消さずに残す」ことをサーバが約束できる印（OPEN4_RESULT_PRESERVE_UNLINKED）も用意された。ただし残すかどうかはサーバしだい

2011	The Twelve-Factor App（Heroku）「ログはイベントの流れとして扱う」	アプリはファイルを管理せず標準出力へ。集めるのは基盤の仕事、という考え方
2012 年 12 月	Spring 3.2 の @ControllerAdvice	例外を 1 か所でまとめて HTTP の状態（500・503 など）に変える仕組み。例外が Controller まで「届けば」使える
2013 ごろ	glibc がエラー文言を「Stale NFS file handle」→「Stale file handle」に変更	NFS 以外でも起きるため。Java の例外の文言はこれ
2014	Log4j 2.0 GA	設計を作り直し。ロールオーバの仕組み（TriggeringPolicy／RolloverStrategy）もこの形に。「ログの失敗でアプリを止めない」が基本方針で、アペンダの ignoreExceptions は既定で true。log4j-api も、呼び出し後に起きた例外を抑える（LOG4J2-1990）
2014～2015	Amazon ECS 発表・一般提供	コンテナを AWS で動かす
2015～2016	Amazon EFS 発表（2015）・一般提供（2016）	AWS の「みんなで使える NFS（v4.0／4.1）」
2017	AWS Fargate	サーバを持たずにコンテナを動かす。タスクごとに独立した VM（ディスクはタスクが止まると消える）
2020 年 4 月	Fargate プラットフォーム 1.4.0 で EFS をマウントできるように	消えるディスクの代わりに、ログや共有データを EFS に置く構成が広がる
2020 年 11 月	EFS 側の技術者が Linux の NFS メーリングリストに「別のクライアントが開いているファイルを消すと、閉じるときに ESTALE」の修正案	EFS は別クライアントに消されたファイルに STALE を返す（knfsd では再現しない）という一次情報
2021 年 3 月	ECS Exec	動いているコンテナの中で調べられる（今回の確認方法で使う）
2021 年 12 月	Log4Shell（CVE-2021-44228）	Log4j2 を最新に保つ運用が当たり前に
2022 年 11 月	Spring Framework 6.0（jakarta.* の名前空間）	Jakarta EE 10 の JBoss EAP 8 系で動く Spring MVC はこの系統
2023 年 9 月	OpenJDK 21（LTS）	今回の Java
2024 年 2 月／2025 年 9 月	JBoss EAP 8.0／8.1	今回のアプリケーションサーバ
2025 年 12 月 15 日	Log4j 2.25.3	今回の版。TLS のホスト名確認の修正（CVE-2025-68161）を含む
2026 年 3～7 月	Log4j 2.25.4・2.26.0・2.26.1・2.25.5	セキュリティ修正（CVE-2026-34477／34478／34480、log4j-api の CVE-2026-49844）。共有ファイルの問題を直す変更は無い
2026 年 5 月	RHEL 9.8（UBI 9.8）	今回のベースイメージ

■ なぜ「ファイルハンドル（引換券）」という仕組みができたのか

NFS は、サーバがクライアントのことを覚えなくてよいように作られた（ステートレス）。サーバが再起動しても、クライアントはそのまま続けられるようにするためだ。そこで、ファイルを「名前」ではなく、サーバが発行する「引換券（ファイルハンドル）」で指すことにした。名前は改名で変わるが、引換券は実体そのものを指すので、改名されても同じ実体に書ける。

その代わり、実体がサーバ側で消されると、クライアントが持っている引換券は使えなくなる。サーバは「その引換券の品物は無い」とだけ返す。これが ESTALE（Stale file handle）だ。クライアントは、ほかのクライアントが何をしたかを知らないので、使おうとして初めて気づく（Linux NFS FAQ）。

■ なぜ「消しても書ける」はずのものが書けないのか

1 台のサーバ（ローカルディスク）では、開いているファイルの名前を消しても、最後の 1 人が閉じるまで中身は残る（Unix の約束）。ログの交換（ロールオーバ）も、この約束に頼って作られている。NFS のクライアントも、自分が開いているファイルを自分で消すときは「.nfsXXXX」という隠し名前に改名して、この約束をまねる（silly rename）。

しかし「別のクライアントが消した」場合は、自分が開いていることをサーバに伝える手段が（NFSv3 では）無く、NFSv4.1 でもサーバが残すと約束した場合だけ残る。EFS は残さず STALE を返す。つまり、複数のタスク（＝複数のクライアント）が同じファイルを消したり置き換えたりする使い方は、Unix の約束の「外」にある。

■ なぜログを「ロールオーバ」するのか

・1 つのファイルが大きくなりすぎると、開く・探す・転送するのがたいへんになる。

・ディスクがあふれると、アプリやサーバが止まる。古いものから消せるように、時間や大きさで区切る。

・「13 時台のログ」のように、時間で探せると調べやすい。

・Log4j2 は、交換の決まり（TriggeringPolicy）と、交換のしかた（RolloverStrategy：改名・圧縮・削除）を分けて設定できる。ただし、交換は「そのファイルを使っている JVM が自分でやる」。ほかの JVM と相談する仕組みは無い。

■ なぜコンテナのログを EFS に置くようになったのか

Fargate のタスクのディスクは、タスクが止まると消える。障害の調査や監査のためにログを残したいので、2020 年に Fargate が EFS をマウントできるようになると、「サーバの /var/log に書いていたものを、そのまま EFS に書く」構成が広がった。オンプレミスの 1 台のサーバでは、1 つのログファイルに書くのは 1 つのプロセスだけだったので、ロールオーバも安全だった。

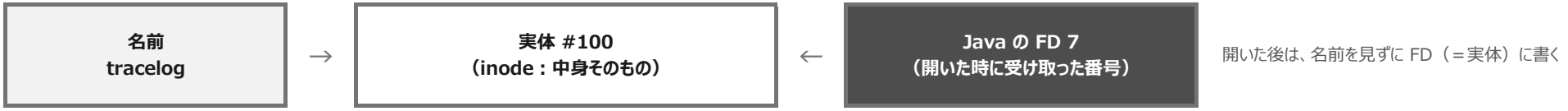
ところがコンテナは、同じイメージ・同じ設定のタスクを何台も並べる（スケールアウト）。設定ファイルにファイル名を固定で書くと、全タスクが同じファイルに書き、全タスクが同じファイルを交換する。「1 台なら正しい設定」が、6 台にしたとたんに事故の元になる。今回の JBoss のログは、タスクごとにディレクトリを分けていたので、この落とし穴を避けられていた。

■ 今回の教訓　共有ストレージ（EFS）に置くのは悪くない。悪いのは「1 つのファイルに複数の書き手」。書き手を 1 つにすれば、EFS の上でもロールオーバは安全に動く（15_検証結果 の C3 で確認）。

04　動作原理：名前・実体・引換券、NFS と EFS の約束、Log4j2 のロールオーバの中身

ここを押さえると「なぜ tracelog だけが」「なぜ 1 タスクだけ正常に」が自然に説明できる。Log4j2 の動きは 2.25.3 のソースコードで確かめた（行番号つき）。

■ 4-1　名前・実体（inode）・FD の三段構え



・名前は、ディレクトリの中の「見出し」にすぎない。改名（rename）は見出しを書き換えるだけで、中身は動かない。

・Java の FileOutputStream は、開いた瞬間に実体への番号（FD）を受け取り、あとはずっとその番号で書く。だれかが名前を付け替えても、Java は元の実体に書き続ける（エラーにはならない）。

・NFS（EFS）では、FD の先にさらに「サーバが発行したファイルハンドル（引換券）」がある。実体がサーバで消されると、この引換券が無効（Stale）になる。

■ 4-2　シンボリックリンクは「開くときの道案内」だけ

/webapp/webapp9mf02/logs → /mnt/logs/front/logs/inter-api（バックは /webapp/webapp9mb02/logs → /mnt/logs/back/logs/inter-api）のリンクは、ファイルを開くとき（と、ロールオーバで改名・作成するとき）に道をたどるのに使われるだけ。開いた後の書き込みは実体に直接行くので、リンクは Stale の原因にならない。ただし、リンクの「行き先」が全タスクで同じだと、全タスクが同じファイルを開くことになる（これが今回の問題）。逆に言えば、書き先（-Diwin.log かリンクの行き先）をタスクごとに変えるだけで、Log4j2 の設定を変えずに直せる。JBoss のログは既に mid/{タスクID} でそうなっている（11_恒久対策）。

■ 4-3　消されたときのふるまい（場所によって違う）

誰が消したか／どこで	開いている側はどうなるか	根拠
同じマシン（ローカルディスク・同じ OS）で消した	そのまま書ける（最後に閉じるまで中身が残る）。ただし名前が無いので、閉じたら中身ごと消える	Unix の約束
同じ NFS クライアント（同じタスク）の中で消した	クライアントが .nfsXXXX という隠し名前に改名して残す（silly rename）。そのまま書ける。閉じると消える	Linux NFS クライアント
別の NFS クライアント（別のタスク）が消した・同じ名前で上書きした【EFS】	次にサーバへ問い合わせたところで NFS4ERR_STALE → ESTALE → Java は IOException: Stale file handle	EFS 側の技術者の LKML 投稿（2020）「別のクライアントが開いているファイルを消すと ESTALE。nfsd では再現しない」

別の NFS クライアントが消した【Linux の knfsd・NFSv4.1】	書き込みはエラーにならない（Log4j2 のエラー表示も 0）。ただし名前が無い実体に書くので、行は黙って消える。stat で問い合わせると Stale になるので、診断ツールでは見つけられる	15_検証結果 C2
別の NFS クライアントが消した【NFSv3】	サーバは誰が開いているか知らないので消す → ESTALE	Linux NFS FAQ A10・15_検証結果 C1

■ EFS とローカルディスクの決定的な違い　ローカルディスクなら、6 つのプロセスが同じファイルを交換し合っても（行の取り違えは起きるが）例外にはならない。EFS では、6 つのタスクが「別々のマシン」なので、他のタスクが消した実体への書き込みが STALE になる。同じ設定でも、置き場所が変わると壊れ方が変わる。

■ 4-4　EFS と Fargate の組み合わせ

・Fargate のタスクは 1 つずつ独立したマイクロ VM で動く。EFS のマウントは Fargate が用意する監督コンテナ（aws-fargate-supervisor）が行い、コンテナには /mnt/logs として見せる。NFS クライアントは各タスクの VM のカーネルにある。だから 6 タスク＝6 つの別々の NFS クライアント。

・同じタスクのフロントとバックは、同じ VM・同じ NFS クライアントを使う（別のディレクトリなので今回は干渉しない）。

・EFS は NFSv4.0／4.1 のサーバ。一貫性は close-to-open（閉じたら書き戻し、開くときに確かめる）。「同期的で、追記ではない書き込み」だけが read-after-write で、追記（ログ）はその保証の外（AWS 公式）。

・EFS のロックは「任意ロック（advisory）」だけで、読み書きはロックを確かめずに行われる（AWS 公式）。Log4j2 のロールオーバはそもそもロックを使わない。

・ECS はタスクの EFS をコンテナの中から付け外しさせない。マウントを付け直したいときは、タスクを入れ替える。

■ 4-5　Log4j2 の RollingFile がやっていること（2.25.3 のソースで確認）

1　ログを 1 件書く（RollingFileAppender.append）

書く前に必ず checkRollover(event) を呼ぶ（RollingFileAppender.java 335～338 行）。交換の時刻かどうかは、ログのイベントが来たときだけ判定する。タイマーは無いので、00 分ちょうどではなく「00 分を過ぎて最初のログ」で交換が起きる。

▼

2　交換の時刻か（TimeBasedTriggeringPolicy.isTriggeringEvent）

イベントの時刻が nextRolloverMillis 以上なら、次の交換時刻を先に進めてから true を返す（TimeBasedTriggeringPolicy.java 130～139 行）。filePattern の最小単位（今回は %d{…HH} の「時」）が変わる時に交換。interval=1・modulate=false・maxRandomDelay=0（既定）。

▼

3　今のファイルを閉じる（RollingFileManager.rollover）

closeOutputStream() で FileOutputStream を閉じる（RollingFileManager.java 622 行）。閉じるのに失敗しても「Unable to close stream」を出して先へ進む。

▼

4　改名する（FileRenameAction）

tracelog を trace.log.<前の時間> に改名する。中身が 0 バイトなら改名せず削除する（renameEmptyFiles=false が既定。FileRenameAction.java 111・194 行）。改名は Files.move(…, ATOMIC_MOVE, REPLACE_EXISTING)（同 209 行）。同じ名前のファイルが既にあれば、黙って上書きする（＝古い方の実体を消す）。

▼

5　新しい tracelog を作って開く（createFileAfterRollover）

new FileOutputStream(ファイル名, append=true) で開く（FileManager.java 251 行）。同じ名前のファイルが既にあれば（ほかのタスクが作っていれば）、それを開いて追記する。

▼

6　書き込む（OutputStreamManager）

8 KB のバッファに書き、immediateFlush=true（既定）なので 1 件ごとに write() する。失敗すると AppenderLoggingException（原因は IOException）を投げ、バッファは捨てる（OutputStreamManager.java 261～267・292～301 行）。開き直しはしない。次に開き直すのは、次の交換のとき。

設定（今回は未指定＝既定値）	既定値	今回の意味
append	true	O_APPEND で開く。NFS では「同時の追記」は保証されない（open(2)・Linux NFS FAQ）
bufferedIO / bufferSize / immediateFlush	true / 8192 / true	1 件ごとに書き出す。失敗した 1 件はバッファごと捨てられる
layout	PatternLayout %m%n	時刻もタスク名も入らない。どのタスクの行かを後から見分けにくい（11 で追加を推奨）
createOnDemand	false	起動時に tracelog を作って開く（全タスクが同じ実体を開く）
locking	false（指定しても RollingFile では使われない）	2.25.3 では Builder にあるが RollingFileManager に渡されない（RollingFileAppender.java 79・144～159 行）。仮に効いても改名・削除は防げない
ignoreExceptions	true	例外はアプリに伝わらず、StatusLogger（標準エラー）に出るだけ。業務処理は続く
DefaultRolloverStrategy max	7	filePattern に %i が無いので、古いファイルを数えて消す処理は空振り（AbstractRolloverStrategy.java 119～120 行）。今回は削除は起きない
TimeBasedTriggeringPolicy	interval=1, modulate=false, maxRandomDelay=0	全タスクが 00 分すぎの最初のリクエストでほぼ同時に交換する
最初の交換時刻の基準	ファイルの作成時刻（取れなければ更新時刻）	途中で起動したタスクは、既存の tracelog の作成時刻から次の交換時刻を決める（RollingFileManager.java 913～932 行）

■ 4-6　同じ JVM なら安全、別の JVM（別タスク）だと危ない理由

Log4j2 は、同じ JVM の中で同じ fileName を使うアペンダには、1 つの RollingFileManager を共有させる。公式マニュアルは「共有すれば交換は 1 回だけ」と書いている。しかしこれは JVM の中だけの話で、6 タスク＝6 つの JVM の間では何も共有されない。つまり交換は 6 回起き、改名先の名前（trace.log.<前の時間>）は 6 回とも同じになる。2 回目以降の改名が、1 回目に保存されたファイルを上書きで消す。

■ 4-7 例外の出方と「ログに出る回数が少ない」理由

JBoss の server.log に出る形（例。StatusLogger が標準エラーへ出し、JBoss が [stderr] として記録する）

```
ERROR [stderr] (default task-12) 2026-10-10T04:00:03.512Z default task-12 ERROR Unable to write to stream /webapp/webapp9mf02/logs/tracelog for appender A4

ERROR [stderr] (default task-12)  org.apache.logging.log4j.core.appender.AppenderLoggingException: Error writing to stream /webapp/webapp9mf02/logs/tracelog

ERROR [stderr] (default task-12)     at org.apache.logging.log4j.core.appender.OutputStreamManager.writeToDestination(OutputStreamManager.java:265)
ERROR [stderr] (default task-12)     ...
ERROR [stderr] (default task-12) Caused by: java.io.IOException: Stale file handle
ERROR [stderr] (default task-12)     at java.base/java.io.FileOutputStream.writeBytes(Native Method)
ERROR [stderr] (default task-12) 2026-10-10T04:00:03.513Z default task-12 ERROR An exception occurred processing Appender A4

ERROR [stderr] (default task-12)  org.apache.logging.log4j.core.appender.AppenderLoggingException: Error writing to stream /webapp/webapp9mf02/logs/tracelog

ERROR [stderr] (default task-12) Caused by: java.io.IOException: Stale file handle

（毎時 00 分すぎ）ERROR [stderr] … ERROR RollingFileManager /webapp/webapp9mf02/logs/tracelog Unable to close stream: java.io.IOException: Stale file handle
```

・エラーの名前に出るのは、リンクの側のパス（/webapp/webapp9mf02/logs/tracelog）。EFS の実体のパスではない。

・Log4j2 の DefaultErrorHandler は、標準エラーがあふれないように「最初の数件（コード上は 4 件）のあとは 5 分に 1 件」に間引く（DefaultErrorHandler.java 31・99～113 行）。1 件の失敗で 2 回呼ばれるので、実際は最初の 2 件ほどの失敗だけが詳しく出て、あとは 5 分に 1 回しか出ない。

・つまり server.log のエラーの回数は、失われたログの件数よりずっと少ない。「エラーが少ないから被害も少ない」とは読めない（15_検証結果 で比較）。

・交換のときに閉じられなかった場合の「Unable to close stream」は間引かれずに出る（AbstractManager の logError）。毎時 00 分すぎに並ぶのが目印。

■ 4-8 交換が無くても、共有ファイルへの追記は安全ではない

・Linux の open(2)：「NFS では、複数のプロセスが同時に O_APPEND で追記すると、ファイルが壊れることがある。NFS には追記の命令が無く、クライアントがまねるので競合は避けられない」。

・Linux NFS FAQ A9：「NFS には原子的な追記が無いので、NFS 上の追記はどの OS でも原子的ではない」。

・つまり、交換の事故が無い時間帯でも、6 タスクの行が重なって上書きされたり、NUL（¥0）で埋まった穴ができたりしうる。

・再現実験（3 回目）では、最初のロールオーバの前の 1 分間、6 タスクが同じ実体に追記していたのに、残ったのは 1 タスク分（158 行）だけだった。各タスクの NFS クライアントは「自分が書いた分」の長さしか知らず、同じ位置に書いては上書きし合い、最後にまとめて書き出したタスクの分だけが残ったため。エラーは 1 件も出ない。

・Log4j2 の公式マニュアルの「append は多くのシステムで追記を原子的にする」は、ローカルディスクの話として読む。

■ 4-9 JBoss の server.log・GC ログ・アクセスログが無事な理由

これらは /mnt/logs/front/logs/inter-api/mid/{タスクID}/ というタスクごとのディレクトリに書かれていて、1 つのファイルに書くのは 1 つの JVM だけ。交換（JBoss の periodic-rotating-file-handler、GC ログの filecount、アクセスログの日付ごとの切り替え）も、自分のファイルに対して 1 回だけ行う。ほかのタスクは触らないので、同じ EFS・同じマウント・しかも tracelog と同じ親ディレクトリの下でも Stale にならない。今回のトレースログのエラーがきちんと server.log に残っているのは、まさにこの違い（書き手の数）の証拠になる。

■ 4-10 例外はどこで止まるのか（Spring MVC の Controller に届かない理由）

1 業務の Controller がログを書く

Spring MVC の Controller（やその先のサービス）が ILGLog4j を通して logger.debug(...) を呼ぶ（SLF4J 経由でも Log4j API 直接でも同じ道）。

▼

2 アペンダの中で失敗する

OutputStreamManager.writeToDestination が FileOutputStream.write() の IOException: Stale file handle を受け、AppenderLoggingException（RuntimeException の仲間）に包んで投げる（OutputStreamManager.java 261～267 行）。

▼

3 AppenderControl が受け止める

AppenderControl.tryCallAppender が受け取り、ErrorHandler（DefaultErrorHandler）に渡して標準エラーへ出す。ignoreExceptions=true なら投げ直さない（AppenderControl.java 158～173 行）。ここで例外は終わる。

▼

4 log4j-api も念のため抑える

AbstractLogger.tryLogMessage は、ログ呼び出しの中で起きた例外をすべて受け止め、LoggingException（AppenderLoggingException など）以外は投げない（AbstractLogger.java 2962～2988 行・LOG4J2-1990）。ignoreExceptions=false のときだけ、AppenderLoggingException が呼び出し元へ届く。

▼

5 Controller には何も届かない

logger.debug(...) はふつうに戻る。Controller もヘルスチェック API も、例外を一度も受け取らない。catch して 503 にすることは、そもそもできない。

A4 の ignoreExceptions	logger.debug() を呼んだ側（業務の Controller）	ヘルスチェック API（ログを書かない場合）	HTTP の結果

true（今回・既定）	何も起きない（ログだけが失われる）	何も起きない	業務もヘルスチェックも 200
false（Failover なし）	AppenderLoggingException が飛ぶ → 処理しなければ業務リクエストが失敗	何も起きない（ログを書かなければ）	業務は 500（Spring MVC の既定。503 にはならない）、ヘルスチェックは 200
false＋Failover で受ける	Failover が受けて標準出力などへ逃がす（例外は外に出ない）	何も起きない	業務もヘルスチェックも 200（ログは逃げ先に残る）

つまり「200 が返る」のは、Controller の実装ミスではなく、Log4j2 の設計（ログの失敗でアプリを止めない）による。ヘルスチェックで気づきたいなら、例外を待つのではなく、ヘルスチェックがトレースログの状態を自分で調べる必要がある（13_ヘルスチェック）。再現実験でも、アプリは 1 件も例外を受け取らずに最後まで動き、標準エラーにだけ Log4j2 のエラーが出た（15_検証結果）。

05　動作イメージ：13:00 の連鎖を 1 コマずつ

T1〜T6 は「13 時を過ぎてロールオーバした順」の番号（どのタスクが何番目になるかは毎時変わる）。#0〜#6 は実体（inode）。○ = 書ける、→保 = 書けるが保存ファイル（前の時間帯の名前）に書いている、× = Stale。連鎖には 2 つの型があり、どちらでも「正しく書けるのは最後の 1 タスクだけ」になる。

■ 2 つの型の分かれ目　後からロールオーバしたタスクが、前のタスクが作ったばかりの tracelog を「0 バイト」と見るか「中身あり」と見るかで分かれる。Log4j2 は 0 バイトなら改名せずに削除し（型 B）、中身があれば同じ名前の保存ファイルを上書きして改名する（型 A）。書いた行は NFS クライアントのキャッシュに少しとどまってから EFS に届くので、ロールオーバが数十〜数百ミリ秒の間に続くと型 B、間があくと型 A になりやすい。再現実験（1 分ごと・6 タスク・3 回・境目 12 回）では、3 回のうち 2 回でデプロイ直後の最初の境目に型 A が 1 回混ざり、それ以外の境目はすべて型 B だった（15_検証結果）。

■ 5-1　型 B「空なら削除」：続けざまにロールオーバしたとき（混んでいる時間帯・実験で多かった型）

時刻	起きたこと（Log4j2 の動き）	名前 tracelog の実体	名前 trace.log.‥-12 の実体	T1	T2	T3	T4	T5	T6
12:59	12 時台の連鎖の結果、tracelog（#0）に正しく書けているのは T4 だけ（12 時台の最後のタスク）。ほかは 12 時台ずっと Stale	#0	（11 時台の保存）	×	×	×	#0 ○	×	×
13:00:00.035	**T1 が交換：自分の（消された）実体を閉じる → Unable to close stream: Stale file handle。tracelog（#0・中身あり）を trace.log.‥-12 に改名。新しい #1**	**#1**	**#0**	**#1 ○**	×	×	**#0 →保**	×	×
13:00:00.051	**T2 が交換：tracelog（#1）は T2 から見て 0 バイト → 改名せずに削除（renameEmptyFiles=false）。新しい #2**	**#2**	**#0**	**#1 ×**	**#2 ○**	×	**#0 →保**	×	×
13:00:00.077	**T3 が交換：#2 も 0 バイトに見える → 削除。新しい #3**	**#3**	**#0**	×	**#2 ×**	**#3 ○**	**#0 →保**	×	×
13:00:00.079	**T4 が交換：#0（保存側）をふつうに閉じる。#3 は 0 バイト → 削除。新しい #4**	**#4**	**#0**	×	×	**#3 ×**	**#4 ○**	×	×

13:00:00.140	T5 が交換：#4 は 0 バイト → 削除。新しい #5	#5	#0	×	×	×	#4 ×	#5 ○	×
13:00:00.267	T6 が交換：#5 は 0 バイト → 削除。新しい #6	#6	#0	×	×	×	×	#5 ×	#6 ○
13:00:01〜13:59	この形が次の 00 分まで続く。T1〜T5 は書くたびに IOException: Stale file handle（ログは捨てられる）	#6	#0	STALE	STALE	STALE	STALE	STALE	CURRENT

時刻は実験の 1 回分（境目からの遅れ 35・51・77・79・140・267 ミリ秒）をそのまま当てはめた。保存ファイル trace.log.‥‥-12 には、12 時台に正しく書けていた 1 タスク（T4）の 12 時台の行だけが、正しい名前で残る。

■ 5-2 型 A「改名で上書き」：間をあけてロールオーバしたとき（空いている時間帯・デプロイ直後の最初の 00 分）

時刻	起きたこと（Log4j2 の動き）	名前 tracelog の実体	名前 trace.log.‥‥-12 の実体	T1	T2	T3	T4	T5	T6
12:59	6 タスクとも同じ実体 #0 に追記している（デプロイ直後で、全タスクが起動時に同じ tracelog を開いた状態）	#0	（なし）	#0 ○	#0 ○	#0 ○	#0 ○	#0 ○	#0 ○
13:00:00	T1 が交換：#0 を閉じ、tracelog→trace.log.‥‥-12 に改名、新しい tracelog #1 を作る	#1	#0	#1 ○	#0 →保	#0 →保	#0 →保	#0 →保	#0 →保
13:00:05	T2 が交換：tracelog（#1・中身あり）を trace.log.‥‥-12 に改名。同じ名前の #0（12 時台の全員分）は上書きで消える。新しい #2	#2	#1	#1 →保	#2 ○	#0 ×	#0 ×	#0 ×	#0 ×
13:00:12	T3 が交換：#0 を閉じる（Unable to close stream）。#2 を改名して #1 を上書きで消す。新しい #3	#3	#2	#1 ×	#2 →保	#3 ○	#0 ×	#0 ×	#0 ×
13:00:20	T4 が交換：#3 を改名して #2 を上書きで消す。新しい #4	#4	#3	×	#2 ×	#3 →保	#4 ○	#0 ×	#0 ×
13:00:31	T5 が交換：#4 を改名して #3 を上書きで消す。新しい #5	#5	#4	×	×	#3 ×	#4 →保	#5 ○	#0 ×
13:00:45	T6 が交換：#5 を改名して #4 を上書きで消す。新しい #6	#6	#5	×	×	×	#4 ×	#5 →保	#6 ○

13:01〜13:59	**この形が次の 00 分まで続く。T1〜T4 は Stale。T5 はエラーを出さずに、12 時台の名前のファイルへ 13 時台の行を書き続ける**	**#6**	**#5**	**STALE**	**STALE**	**STALE**	**STALE**	**ARCHIVE**	**CURRENT**

・実体が「消える」のは、改名先が同じ名前（trace.log.‥-12）だから。Files.move の REPLACE_EXISTING（と POSIX の rename）は、既にある同名のファイルを置き換える。

・途中の「#0 ✕」は、自分の交換までのあいだに書けば Stale になる、という意味。交換が来ると新しいファイルを開くので、そこで抜け出す。

・上の表は、すべての交換が型 A になったときの形（このときは最後から 2 番目が ARCHIVE）。

・実験では、デプロイ直後の最初の境目で「2 番目のタスクだけが型 A、3 番目以降は型 B」という混ざった形になった。2 番目のタスクの NFS クライアントが、起動時に開いた tracelog の大きさを覚えていて「中身あり」と判断したためで、ARCHIVE になったのは最初にロールオーバしたタスクだった（1・2 回目とも同じ。3 回目は最初から型 B だけで、ARCHIVE は出なかった）。

・型 A と型 B がどう混ざっても、最後にロールオーバしたタスクだけが CURRENT になる。

■ 5-3 13 時台が終わったとき、何が残っているか

	型 B（空なら削除）	型 A（改名で上書き）
tracelog	最後のタスク（T6）の 13 時台だけ。14:00 の連鎖で trace.log.‥-13 として保存される（次も型 B なら正しく残る）	最後のタスク（T6）の 13 時台だけ。14:00 の連鎖が型 A だと、保存された直後に上書きで消える
trace.log.‥-12	12 時台の最後のタスク（T4）の 12 時台の行だけ（名前と中身が合っている）	T5 の「13 時台」の行だけ（名前は 12 時台。1 時間ずれ）。12 時台の本物は消えた
そのほか	T1〜T5 の 13 時台の行はどこにも無い	T1〜T4 の 13 時台の行はどこにも無い
残る割合	全体の約 1/6（1 タスク分）	全体の約 1/6（1 タスク分・名前がずれる）

■ ここからわかる見分け方　trace.log.YYYY-MM-DD-HH の中身が 1 タスク分しか無い（行数がふだんの 1/6 ほど・同じスレッド名や同じ利用者の行だけ）、または更新時刻が名前の時間の「次の 1 時間の終わりごろ」になっていれば、この連鎖が起きている。どちらの型でも、残るのは約 1/6。

■ 5-4 1 つのタスクの中で、例外が出るまで（型 B の T1 の視点）

時刻	T1 の Java（Log4j2）	T1 のカーネル（NFS クライアント）	EFS（NFS サーバ）	ほかのタスク
13:00:00.035	rollover：古い実体を閉じ、tracelog を改名し、新しい tracelog（#1）を開いて 1 行書く	CLOSE・RENAME・CREATE。書いた行はまずキャッシュへ	名前を付け替え、#1（0 バイト）を作る	

13:00:00.051	（#1 に書いているつもり）	（行はまだキャッシュの中）	T2 からの削除で #1 を消す（T1 が開いていても、別クライアントなので残さない）	T2：tracelog の大きさ 0 → 削除
13:00:03	リクエストのログ：FileOutputStream.write()	追記なのでファイルの大きさを確かめる（GETATTR #1）／たまった行を書き出す（WRITE #1）	NFS4ERR_STALE（その引換券の品物は無い）	
13:00:03	write() が ESTALE(116) → IOException: Stale file handle → AppenderLoggingException	ESTALE を返し、この実体を「Stale」と覚える		
13:00:03	DefaultErrorHandler → StatusLogger → 標準エラー → JBoss の server.log に ERROR [stderr]。リクエストの処理は続く（ignoreExceptions=true）			
13:00:04〜13:59	書くたびに失敗。server.log に出るのは最初の 2 件ほどと、そのあと 5 分に 1 件だけ	すぐ ESTALE		
14:00:00.x	次の rollover：#1 を閉じる（Unable to close stream）。tracelog を改名／削除し、新しい tracelog を開く → いったん直る			ほかのタスクも交換。また連鎖

■ 5-5 仮説 1：動いている最中に EFS のファイルシステムを作り直した場合

時刻	6 タスクの Java	各タスクのカーネル（NFS クライアント）	EFS	作業者
t0	tracelog・server.log・gc.log・アクセスログに書いている	古い FS の引換券を持っている	古い FS（fs-old）	
t1	ログを書くスレッドが止まり始める	マウントターゲットが消え、応答が無いので待ち続ける（hard マウント）	マウントターゲット削除	DeleteMountTarget
t2			fs-old を削除、fs-new とマウントターゲットを作成	DeleteFileSystem → CreateFileSystem → CreateMountTarget
t3（ふつう）	ログを書く処理が返ってこない。ヘルスチェックが応答しなければ ECS がタスクを入れ替える	新しいマウントターゲットは別の IP・別の名前。古い宛先を待ち続ける		

t3（同じ IP のとき）	**全ファイルの書き込みが失敗。交換時の作り直しも FileNotFoundException（Stale）。ls /mnt/logs も失敗**	**新しい FS は古い引換券を知らない → bad file handle／Stale（ルートも）**		
t4	**タスクを入れ替えるまで直らない（Fargate ではマウントし直せない）**		**新しいタスクは fs-new をマウント（タスク定義が fs-new を指していれば）**	**サービスの強制デプロイ**

AWS 公式のトラブルシュートにも「FS とマウントターゲットを消し、同じ IP で新しい FS とマウントターゲットを作ると、古いマウントで bad file handle になる。アンマウントしてマウントし直すと直る」とある。再現実験（C4）でも、作り直した直後から 6 タスクすべてで ls /mnt/logs が Stale になり、タスクごとの server.log まで書けなくなった。壊れるのはマウント全体で、tracelog だけが壊れることは無い。

■ 5-6 2 つの仮説の「見え方」の違い

仮説 2（共有ファイル＋ロールオーバ）のとき	仮説 1（FS の作り直し）のとき
壊れるのは共有ファイル（tracelog）だけ	/mnt/logs の下の全ファイルが壊れる
タスクごとのログ（server.log など）は無事	server.log・GC・アクセスログも同時に書けない
6 タスク中 4〜5 タスクがエラー、正しく書けるのは 1 タスク	全タスクが同時に壊れる
毎時 00 分すぎに始まり、正常なタスクが毎時入れ替わる	作り直した瞬間から（時刻と関係ない）
ls /mnt/logs は正常	ls /mnt/logs も Stale・bad file handle・応答なし
再起動でしばらく直るが、次の 00 分でまた起きる	タスクを入れ替えれば直り、二度と起きない
trace.log.* の中身が 1 タスク分だけ（名前の時間がずれることも）	CloudTrail に DeleteFileSystem・CreateFileSystem が残る

06　6 タスクのうち 1 タスクだけ正常、はありうるか → ありうる（毎時の「ふつうの姿」）

00 分すぎに「最後にロールオーバしたタスク」だけが、次の 00 分まで tracelog に正しく書ける。ほかのタスクは Stale（または、エラーを出さずに前の時間帯の保存ファイルへ書く）。どのタスクが正常になるかは毎時入れ替わる。

■ 答え　ありうる。今の設定のままなら、毎時ほぼ必ず「正しく書けるのは 1 タスク」になる。再現実験 C1（6 タスク・1 分ごとのロールオーバ・EFS と同じく「消された実体に Stale を返す」NFS サーバ）を 3 回行い、12 回の境目のうち 12 回で、正しく書けたのは「その分の最後にロールオーバしたタスク」1 つだけだった。本番と同じ配置の 3 回目では、書いた 5998 行のうち、正しいファイルに残ったのは 1020 行（17.0%）、別の時間帯のファイルに入ったのが 0 行（0.0%）、消えたのが 4978 行（83.0%）。

■ 6-1　規則：N タスクが同じ tracelog を使うと、その時間帯はこうなる

立場（その時間の 00 分すぎ）	その時間帯の状態	エラー	その時間帯のログの行方
最後にロールオーバしたタスク	CURRENT（tracelog の今の実体に書いている）	出ない	tracelog に正しく入る。次の 00 分に最初のタスクが trace.log.<この時間> に改名して保存する（次の連鎖が型 A だと、直後に上書きで消える）
自分が作った新しい tracelog を、次のタスクに（削除ではなく）改名で保存側へ回され、その保存ファイルがその後上書きされなかったタスク（型 A が起きたときだけ 1 つ出る。実験ではデプロイ直後の最初の境目で、最初にロールオーバしたタスクがこれになった）	ARCHIVE（前の時間帯の名前のファイルに書いている）	出ない	trace.log.<前の時間> に入る。名前と中身の時間が 1 時間ずれる
それ以外のタスク	STALE（実体が EFS 上で消されている）	書くたびに IOException: Stale file handle（server.log に出るのは最初の 2 件ほどと 5 分に 1 件）	どこにも残らない（次の 00 分の自分の交換まで約 1 時間）

■ 6-2 「正常」には 3 つの意味がある

「正常」の意味	当てはまるタスクの数	説明
例外（Stale file handle）が出ていない	1〜2	CURRENT と ARCHIVE。ARCHIVE は「書けているが、別の時間帯のファイル」なので、エラーが無くても正常とは言えない

いま tracelog に正しく書けている	1（まれに 2）	最後にロールオーバしたタスク。2 タスクがほぼ同時に交換して、同じ新しい tracelog を開いたときだけ 2
その時間帯のログが、正しい名前で最後まで残る	0〜1	型 B（空なら削除）が続けば CURRENT の 1 タスク分が残る。型 A（改名で上書き）なら CURRENT の分は次の 00 分に消え、ARCHIVE の分が 1 時間ずれた名前で残るだけ

■ 6-3 形がくずれるパターン（1 タスクにならないとき）

条件	何が起きるか	正しく書けるタスク
2 タスクがほぼ同時（ミリ秒以内）にロールオーバした	後のタスクの削除・改名が空振りし、2 タスクが同じ新しい tracelog を開くことがある	最後の 2 つ（2 タスク）
00 分を過ぎてもログを書かないタスクがある（リクエストが来ない）	そのタスクは交換しない。前の時間の状態（STALE なら STALE）のまま	変わらず 1。ただし交換しないタスクは次の連鎖に遅れて参加し、そのとき誰かを Stale にする
途中でタスクが入れ替わった（デプロイ・オートスケール・異常終了からの自動復旧）	新しいタスクは起動時に今の tracelog を開く → 次の 00 分までは CURRENT と同じ実体に書ける	一時的に 2（CURRENT＋新しいタスク）
起動したタスクから見て tracelog の作成時刻が前の時間帯だった	最初のログで即ロールオーバする（00 分を待たない）→ 連鎖がその場で起きる	1（起動したタスクが引き金）
デプロイ中で新旧タスクが重なっている	最大 12 タスクが同じファイルを使う。連鎖の参加者が増える	1。STALE が増える
深夜などロールオーバの間隔が長い	型 A（改名で上書き）になりやすい → ARCHIVE が出て、保存ファイルの時間がずれる	1（＋エラーの出ない ARCHIVE が 1）
デプロイ直後の最初の 00 分	全タスクが同じ実体を開いた状態から始まり、型 A が混ざることがある（実験 1・2 回目は最初の境目で ARCHIVE が 1 つ出た。3 回目は出なかった）	1（＋ARCHIVE 0〜1）
デプロイ直後から最初の 00 分まで（交換の前）	全タスクが同じ実体に追記するので、エラーは出ない。しかし NFS の追記は上書きし合い、残るのはほぼ 1 タスク分（実験 3 回目：6 タスクのうち 1 タスク分だけ）	エラー 0 でも、正しく残るのは実質 1

■ 6-4 再現実験での確認（C1：境目ごとの実際の状態。行の行き先から判定）

境目	ロールオーバの順番（左が先）	CURRENT	ARCHIVE	STALE	正しく書けたタスク
09:52:00（UTC）	T1 → T2 → T4 → T5 → T6 → T3	1	0	5	T3（順番の最後: T3）
09:53:00（UTC）	T5 → T6 → T2 → T4 → T1 → T3	1	0	5	T3（順番の最後: T3）
09:54:00（UTC）	T1 → T4 → T2 → T5 → T3 → T6	1	0	5	T6（順番の最後: T6）
09:55:00（UTC）	T5 → T3 → T2 → T1 → T6 → T4	1	0	5	T4（順番の最後: T4）

どの境目でも、正しく書けたのは「その分の最後にロールオーバしたタスク」だった。予想どおり、正常なタスクは毎回入れ替わった。

■ 6-5 ELB とお客さまへの影響

・ELB は 6 タスクにほぼ均等にリクエストを振り分ける。正しく書けるのが 1 タスクなら、リクエストのおよそ 5/6 のトレースログが失われる。

・業務処理そのものは成功している（Log4j2 は ignoreExceptions=true で例外をアプリに伝えない）。利用者からは見えず、困るのは調査・監査のときに「ログが無い」こと。

・ヘルスチェックは 200 を返し続けるので、振り分けは止まらない（止めるべきかは 13_ヘルスチェック）。

・server.log のエラーの件数は、Log4j2 の間引きで実際の失敗よりずっと少ない。件数で被害を見積もらないこと（実験：4 分間に 750〜1,000 行が消えたタスクで、書き込み失敗の表示は 2 件）。

■ 6-6 自分の環境で確かめる（08 の C-2）

6 タスクで tools/tracelog-fd-check.sh を実行したときの読み方（例。毎時 02 分以降に実行する）

```
VERDICT=STALE   host=ip-10-0-1-11... fd=512 fd_inode=- ...          ← T1: 次の 00 分まで書くたびに Stale
VERDICT=STALE   host=ip-10-0-2-12... fd=508 fd_inode=- ...          ← T2
VERDICT=CURRENT host=ip-10-0-1-13... fd=511 fd_inode=8615 cur_inode=8615 name=tracelog   ← 正しく書けている唯一のタスク
VERDICT=STALE   host=ip-10-0-2-14... fd=509 fd_inode=- ...          ← T4
VERDICT=ARCHIVE host=ip-10-0-1-15... fd=510 fd_inode=8611 name=trace.log.2026-10-10-12  ← エラーは出ないが前の時間帯のファイル
VERDICT=STALE   host=ip-10-0-2-16... fd=507 fd_inode=- ...          ← T6
```

CURRENT が 1（まれに 2）、残りが STALE（と ARCHIVE が 0〜1）なら、この章の現象が起きていると確定できる。次の時間にもう一度実行して、CURRENT のタスクが入れ替わっていれば、なお確か。

07　原因一覧：可能性の高い順に全 10 項目

★ はこの環境（ECS Fargate・EFS の共有パス・Log4j2 2.25.3・6 タスク・JBoss のログは無事）での見込み。1〜2 位は同じ根（1 ファイル複数書き手）。

順位	原因	可能性	今回の観察との合い方
1	6 タスクが同じ tracelog に書き、各 JVM の Log4j2 が毎時ロールオーバ	★★★★★	すべて合う
2	デプロイ中の新旧タスクの重なり・途中で起動したタスクの即時ロールオーバ	★★★★☆	合う（1 を強める）
3	ほかの書き手（バック・別 WAR・別環境・別サービス）	★★★☆☆	要確認（書き手が増えると悪化）
4	ログの片付けが動いているタスクのファイルを消した	★★★☆☆	片付けがあれば部分的に合う
5	EFS 上のディレクトリの作り直し	★★☆☆☆	一時的な引き金としてはありうる
6	EFS のファイルシステムの作り直し（仮説 1）	★☆☆☆☆	合わない（マウント全体が壊れる）
7	アクセスポイント／rootDirectory の作り直し	★☆☆☆☆	合わない
8	DataSync・復元・同期による置き換え	★☆☆☆☆	ふつうは合わない
9	ネットワーク切断・リース切れ・クライアントの不具合	☆☆☆☆☆	合わない（Stale ではない）
10	EFS 側の障害・メンテナンス	☆☆☆☆☆	合わない

■ 実際によく起きる場面（この環境に限らず、Stale file handle の典型）

よくある場面	何が起きているか	今回との関係
複数のサーバ・コンテナが、共有ストレージ（NFS・EFS）の同じログファイルに書き、それぞれがロールオーバする	後から交換した側が、先の側のファイルを改名・削除する（Log4j2・Logback・JBoss のファイルハンドラ・logrotate のどれでも起きる）	今回そのもの（1 位）
別のホストの片付けジョブ（cron の find -delete、logrotate、Lambda）が、まだ開かれているファイルを消す	消された側のホストの書き込み・読み込みが Stale になる	片付けがあれば（4 位）
デプロイ・リリースのスクリプトが、共有ディレクトリを rm -rf して作り直す	そのディレクトリの中を開いていたプロセスが全部 Stale	5 位
シェルやプロセスのカレントディレクトリが、別のホストから消される	ls や pwd が「Stale file handle」になる（ログ以外でよく見る）	調査中の自分のシェルでも起きうる

NFS サーバ側の作り直し（エクスポートの付け替え、FS の作り直し、復元で置き換え）	マウント全体が Stale。再マウント（＝コンテナではタスクの入れ替え）まで直らない	6〜8 位（仮説 1）
CI のワークスペースやキャッシュを共有ストレージに置き、別のエージェントが消す	ビルドの途中で Stale	同じ形（共有して、別の誰かが消す）
Kubernetes（EKS）で複数の Pod が EFS の同じファイルに書く	ECS と同じ。Pod ごと（ホストごと）に別の NFS クライアント	同じ形

共通するのは「共有ストレージの上で、ある実体を開いている誰かがいるのに、別の誰か（別のホスト）がその実体を消す・置き換える」こと。同じホストの中なら Unix の約束で守られるが、別のホスト（別の NFS クライアント）からの削除は守られない。

■ 各原因の詳細（仕組み・見分け方・緊急／恒久／運用）

▸ 1 位　6 タスクが同じ tracelog に書き、各 JVM の Log4j2 が毎時ロールオーバする　★★★★★

項目	内容
仕組み	交換は JVM ごとに独立。2 番目以降のタスクが、前のタスクの新しい tracelog を「空なら削除」し、中身があれば同じ名前の保存ファイルを「上書き」して改名する。消された実体を開いていたタスクに EFS が STALE を返す（04・05）
よくある場面	1 台のサーバ用に作った log4j2.xml（固定のファイル名）を、そのままコンテナで複数タスクに展開したとき
今回の観察との合い方	すべて合う：tracelog だけ・タスクごとのログは無事・一部のタスクだけ・1 タスクだけ正常・正常なタスクが入れ替わる
確かめ方	08 の C-1〜C-4（エラーの時刻が毎時 00 分すぎ、FD の判定が CURRENT 1・残り STALE、trace.log.* が 1 タスク分）
緊急	書き先をタスクごとに分けてデプロイ（10）。再起動だけでは直らない
恒久	1 ファイル 1 書き手（リンク先・-Diwin.log・${hostName} のどれかでタスクごとに）。正本を CloudWatch Logs に（11）
運用	共有ファイルのままの間は、デプロイを 00 分の前後に重ねない（12）

▸ 2 位　デプロイ中の新旧タスクの重なり・途中で起動したタスクの即時ロールオーバ　★★★★☆

項目	内容
仕組み	ローリングデプロイでは新旧が重なり、最大 12 タスクが同じファイルを使う。起動したタスクは tracelog の作成時刻を基準に次の交換時刻を決めるので、前の時間帯に作られたファイルなら最初のログで即ロールオーバし、その場で連鎖を起こす
よくある場面	「デプロイして起動中に起きた」という観察。デプロイ直後の最初の 00 分に型 A（上書き）が起きやすい
今回の観察との合い方	合う（1 位を強める要因）。「デプロイ・起動中に起きた」という仮説 1 の観察も、実はこちらで説明できる
確かめ方	ECS のサービスイベント・デプロイの時刻（aws ecs describe-services）と、エラーの始まりの時刻を並べる
緊急	1 位と同じ
恒久	1 位と同じ（書き手を分ければ、重なっても問題にならない）

運用	デプロイは 00 分の 5 分後〜55 分の間に始めて終える（共有ファイルのままの間）

▸ **3 位　ほかの書き手が同じパスに書いている（バックコンテナ・別の WAR・別環境・別サービス）　★★★☆☆**

項目	内容
仕組み	書き手が増えるほど連鎖が起きる。同じ JVM の中でも、2 つの WAR がそれぞれ log4j-core を持っていれば書き手は 2 つ（同じ NFS クライアントなので Stale ではなく .nfsXXXX になって消える）
よくある場面	バックの -Diwin.log がフロントと同じ場所を向いている、テスト環境と本番が同じ EFS・同じパス、別サービスが同じアクセスポイントを使う
今回の観察との合い方	要確認。書き手が増えるほど連鎖が長くなり、STALE のタスクが増える（バックの -Diwin.log・同じ JBoss の別 WAR を C-8 で確認）
確かめ方	08 の C-8（各コンテナの実際の -Diwin.log とリンク先）、C-9（EFS の ClientConnections が想定のタスク数より多くないか）
緊急	余計な書き手を止める／書き先を分ける
恒久	パスの命名規則（環境/サービス/コンテナ/タスク ID）を決め、レビューで確認する
運用	環境ごとに EFS（またはアクセスポイントと rootDirectory）を分ける

▸ **4 位　ログの片付け（find -delete・Lambda・cron・手作業 rm）が動いているタスクのファイルを消した　★★★☆☆**

項目	内容
仕組み	日数や名前で消すと、長く書かれていない（しかしまだ開かれている）ファイルや、動いているタスクのディレクトリを消すことがある → そのタスクは Stale
よくある場面	「7 日より古い trace.log.* を消す」「容量が増えたので手で消した」
今回の観察との合い方	片付けがあれば部分的に合う（片付けの時刻に始まる。毎時の入れ替わりは説明できない）
確かめ方	片付けの仕組みの有無と実行時刻、EFS の MetadataIOBytes の山、作業記録
緊急	片付けを止める。影響タスクは次の交換か入れ替えで直る
恒久	片付けは「止まったタスクのディレクトリ」だけ。動いているタスク（ECS の一覧）のものは消さない
運用	手作業で EFS 上のログを消さない。消すときは手順書とチェックリスト（12）

▸ **5 位　EFS 上のディレクトリを消して作り直した（テストの初期化・デプロイスクリプト・手作業）　★★☆☆☆**

項目	内容
仕組み	rm -rf logs && mkdir logs のように作り直すと、中のファイルを開いていた全タスクが Stale。開き直すまで直らない
よくある場面	「テスト用に EFS を作り直した」が、実はディレクトリの作り直しだった、というケース
今回の観察との合い方	一時的な引き金としてはありうる。ただし毎時の入れ替わりや「1 タスクだけ正常」は説明できない。JBoss のディレクトリも消していれば JBoss も壊れる

確かめ方	作業記録・スクリプト。ディレクトリの inode（ls -id）が以前と変わっているか
緊急	作り直したディレクトリを使う全タスクを入れ替える（次の交換で直ることもある）
恒久	起動スクリプトで消さない（mkdir -p だけ）
運用	動いている間は作り直さない。作り直すなら 0 タスクにしてから

▸ 6 位　EFS のファイルシステム（やマウントターゲット）を、タスクが動いたまま作り直した（仮説 1）　★☆☆☆☆

項目	内容
仕組み	古い FS の引換券はすべて無効。マウントターゲットが消えると、クライアントは待ち続ける（hard マウント）。同じ IP で作り直すと bad file handle／Stale（AWS 公式）
よくある場面	テストで Terraform／CloudFormation の destroy → apply をサービスを止めずに行った
今回の観察との合い方	合わない：JBoss のログも同じ EFS なら一緒に壊れる・全タスク同時・時刻と無関係（実験 C4 で確認）
確かめ方	CloudTrail の DeleteFileSystem／DeleteMountTarget／CreateFileSystem の時刻、タスクの中で ls /mnt/logs
緊急	全タスクを入れ替える（タスク定義が新しい FS を指していること）
恒久	EFS の削除保護の運用（IaC で prevent_destroy など）、FS を作り直す手順の整備
運用	作り直すときは desiredCount=0 → 作り直し → タスク定義の更新 → 起動

▸ 7 位　アクセスポイント／rootDirectory の削除・作り直し　★☆☆☆☆

項目	内容
仕組み	マウントの根元が別物になる → 根元から Stale
よくある場面	アクセスポイントを作り直した、rootDirectory に当たるディレクトリを消した
今回の観察との合い方	合わない（マウント全体が壊れる）
確かめ方	CloudTrail の DeleteAccessPoint／CreateAccessPoint、ls /mnt/logs が Stale か
緊急	タスクの入れ替え
恒久	アクセスポイントを IaC で管理し、作り直しを手順化
運用	6 と同じ

▸ 8 位　AWS DataSync・復元・同期ツールが同じパスのファイルを置き換えた　★☆☆☆☆

項目	内容
仕組み	置き換え（上書きコピー・改名での差し替え）は「消して作り直す」と同じ → 開いているタスクが Stale
よくある場面	別環境からのログ同期、上書きの復元

今回の観察との合い方	ふつうは合わない（実行の時刻だけに起きる）
確かめ方	DataSync のタスク履歴、AWS Backup の復元ジョブ（EFS の復元は既定で別ディレクトリに戻す）
緊急	同期を止める
恒久	動いているログディレクトリを同期先にしない
運用	復元先は新しいディレクトリにする

▸ 9 位　ネットワーク切断・NFS のリース切れ・クライアントの不具合　☆☆☆☆☆

項目	内容
仕組み	接続が切れても、ふつうは自動で再接続し、引換券は使える。長い切断ではリースが切れて EIO などになる
よくある場面	AZ の障害、セキュリティグループの変更
今回の観察との合い方	合わない（Stale ではなく、応答なし・EIO になる。全ファイルに出る）
確かめ方	同じ時刻にほかのファイルも遅い・失敗していないか、EFS のメトリクス
緊急	原因の切り分け後、タスクの入れ替え
恒久	—
運用	—

▸ 10 位　EFS 側の障害・メンテナンス　☆☆☆☆☆

項目	内容
仕組み	EFS は引換券が保たれるように作られている（サーバの入れ替えでも使い続けられる）
よくある場面	—
今回の観察との合い方	合わない
確かめ方	AWS Health Dashboard、サポートへの問い合わせ
緊急	—
恒久	—
運用	—

08　確認方法：ログ・EFS 上のファイル・タスクの中・AWS の記録

C-1〜C-4 で仮説 2 を確定、C-5〜C-7 で仮説 1 を否定、C-8〜C-10 で「ほかの書き手」と再現を確かめる。読むだけの操作から順に並べた。

■ C-1　エラーの時刻と並び（JBoss の server.log：タスクごと）

各タスクの server.log で（ECS Exec のシェル、または EFS をマウントした場所で）

```
grep -n -E 'Stale file handle|Unable to write to stream|Unable to close stream|Failed to create file after rollover' server.log | cut -c1-200
# 時刻だけを並べて、毎時 00 分すぎに固まっているかを見る
grep -E 'Unable to close stream' server.log | awk '{print $1, $2}'
```

・「Unable to close stream: java.io.IOException: Stale file handle」が毎時 00 分 00 秒台に並ぶ → ロールオーバのときに、自分の実体が既に消されていた（＝前の時間ずっと Stale だった）証拠。間引かれずに毎回出る。

・「Unable to write to stream … Caused by: java.io.IOException: Stale file handle」は、最初の 2 件ほどのあとは 5 分に 1 件しか出ない（DefaultErrorHandler の間引き）。件数ではなく「出ているか」で見る。

・6 タスクを並べて、同じ時間に「Unable to close stream」が出ていないタスクが 1 つだけあれば、それがその前の時間の CURRENT。

■ C-2　6 タスクの FD を判定する（tools/tracelog-fd-check.sh）

ECS Exec で各タスクのフロントコンテナに入り、ツールを実行する（ディレクトリは Java の -Diwin.log から自動で読む）。毎時 02 分以降に実行する（ロールオーバ直後の 1 分ほどは NFS の名前のキャッシュで ARCHIVE を CURRENT と見誤ることがある。再現実験でもロールオーバ 25 秒後の判定で数回あった）。STALE の判定は EFS の返事そのものなので、いつでも確か。数えるのはそのディレクトリ直下の tracelog・trace.log.* だけで、対策後に同じ mid/{タスクID} に置いた server.log などは数えない。

期待される形（仮説 2 なら）

```
CURRENT が 1（まれに 2）、残りは STALE（ARCHIVE が 0〜1）
1 時間後にもう一度実行すると、CURRENT のタスクが入れ替わっている
```

■ C-3　EFS 上のファイルを見る（どこからでも。読むだけ）

共有ディレクトリの一覧と行数（タスクの中、または EFS を読み取り専用でマウントした EC2 から）

```
ls -li --time-style=full-iso /mnt/logs/front/logs/inter-api/ | grep -E 'tracelog|trace[.]log[.]' | tail -n 30
for f in /mnt/logs/front/logs/inter-api/trace.log.2026-10-10-*; do printf '%s %s¥n' "$(wc -l < "$f")" "$f"; done
head -n 3 /mnt/logs/front/logs/inter-api/trace.log.2026-10-10-12; tail -n 3 /mnt/logs/front/logs/inter-api/trace.log.2026-10-10-12
```

・保存ファイルの行数が、ふだんの全体量の 1/6 ほどしかない → 1 タスク分しか残っていない。

・保存ファイルの更新時刻が、名前の時間の「次の 1 時間の終わりごろ」→ 型 A（上書き）で ARCHIVE のタスクが書いた。

・行に時刻やタスク名が無い（レイアウト %m%n）ので、メッセージの中身（利用者 ID・スレッド名など）で 1 タスク分かを見分ける。見分けられない場合は 11 のレイアウト追加を先に行う。

■ C-4 ロールオーバの時刻とエラーの関係

・TimeBasedTriggeringPolicy は「00 分を過ぎて最初のログ」で交換する。リクエストが多い時間帯は全タスクが数百ミリ秒以内に交換する（型 B）。

・C-1 の「Unable to close stream」の時刻が各タスクで 00:00.0x〜00:0x に並び、その直後から別のタスクで書き込みエラーが出ていれば確定。

■ C-5 マウント全体が生きているか（仮説 1 の否定）

各タスクの中で

```
ls /mnt/logs >/dev/null && echo MOUNT_OK
stat -c '%i %n' /mnt/logs /mnt/logs/front/logs/inter-api /mnt/logs/front/logs/inter-api/mid
grep ' /mnt/logs ' /proc/mounts
```

ls が「Stale file handle」「bad file handle」になる、または返ってこない → マウント単位の問題（仮説 1・原因 5〜7）。正常なら仮説 1 ではない。

■ C-6 JBoss のログ（mid/{タスクID}）が書け続けているか（比較の前提）

各タスクの中で

```
PID=$(for p in /proc/[0-9]*; do [ "$(readlink $p/exe 2>/dev/null | sed 's#.*/##')" = java ] && echo ${p#/proc/}; done | head -1)
tr '¥0' '¥n' < /proc/$PID/cmdline | grep -E 'jboss.server.log.dir|Xlog:gc|iwin.log'
ls -l --time-style=full-iso /mnt/logs/front/logs/inter-api/mid/ | tail -n 8
ls -l --time-style=full-iso /mnt/logs/front/logs/inter-api/mid/<自分のタスクID>/
```

JBoss の server.log・GC・アクセスログは /mnt/logs/front/logs/inter-api/mid/{タスクID}/、つまり tracelog と同じ親ディレクトリの下にある。それが今この瞬間も更新されているなら、EFS・マウント・親ディレクトリは健全で、壊れているのは共有ファイル（tracelog）だけ、と言える。仮説 1（FS の作り直し）や原因 5（ディレクトリの作り直し）なら、ここも同時に壊れているはず。

■ C-7 AWS の記録（作り直し・デプロイの時刻）

CloudTrail（管理イベントは 90 日分を検索できる）・ECS

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventSource,AttributeValue=elasticfilesystem.amazonaws.com ¥
  --start-time 2026-10-01T00:00:00Z --query 'Events[].[EventTime,EventName,Username]' --output table
# DeleteFileSystem / CreateFileSystem / DeleteMountTarget / CreateMountTarget / DeleteAccessPoint / CreateAccessPoint を探す
aws ecs describe-services --cluster <クラスタ> --services <サービス> --query 'services[0].events[0:30].[createdAt,message]' --output table
```

■ C-8 設定の実際の値（書き手の数の確認）

フロントとバックの両方で

```
PID=$(for p in /proc/[0-9]*; do [ "$(readlink $p/exe 2>/dev/null | sed 's#.*/##')" = java ] && echo ${p#/proc/}; done | head -1)
tr '¥0' '¥n' < /proc/$PID/cmdline | grep -E 'iwin.log|log4j'
ls -l /webapp/webapp9mf02/ /webapp/webapp9mb02/ 2>/dev/null
readlink -f /webapp/webapp9mf02/logs /webapp/webapp9mb02/logs 2>/dev/null
ls /opt/*/standalone/deployments/ 2>/dev/null   # JBoss に置かれている WAR の数（場所はイメージに合わせる）
```

・フロント 6 タスクの -Diwin.log（またはリンク先）がすべて /mnt/logs/front/logs/inter-api → 6 書き手（今回の前提）。

・バックの -Diwin.log が /mnt/logs/back/logs/inter-api なら、フロントとは別。ただしバックも 6 タスクで同じ tracelog に書いていれば、バック側でも同じ連鎖が起きる（バックの server.log も確認）。

・バックの書き先がフロントと同じディレクトリ → 12 書き手（原因 3）。

・同じ JBoss に複数の WAR があり、それぞれが WEB-INF/lib に log4j-core を持ち、同じ tracelog を設定している → 1 タスクの中でも書き手が 2 つ以上（原因 3）。

■ C-9 ほかの書き手がいないか（EFS の接続数）

EFS の ClientConnections（Sum を分数で割ると平均の接続数）

```
aws cloudwatch get-metric-statistics --namespace AWS/EFS --metric-name ClientConnections ¥
  --dimensions Name=FileSystemId,Value=fs-xxxxxxxx --statistics Sum --period 60 ¥
  --start-time 2026-10-10T03:00:00Z --end-time 2026-10-10T04:00:00Z
```

1 タスクで 1 接続が目安（Fargate はタスクごとに 1 つの NFS クライアント）。6 タスクの時間帯に 6 より大きければ、ほかのタスク・環境・EC2 が同じ EFS をマウントしている。

■ C-10 テスト環境で再現する（確実に確かめたいとき）

・テスト環境で 2〜3 タスクにし、log4j2.xml の filePattern だけを %d{yyyy-MM-dd-HH-mm}（1 分ごと）にしてデプロイする。

・数分待ち、C-1・C-2・C-3 を行う。毎分、正しく書けるのが 1 タスクだけになれば、本番と同じ現象。

・このリポジトリの lab/run_lab6.sh は、同じことを Docker だけで行う（15_検証結果）。

09　切り分け：はい／いいえで原因を絞る

上から順に答える。最初の 2 問で「マウント単位」か「ファイル単位」かが分かれ、3 問目以降でファイル単位の原因を絞る。

Q1　失敗しているタスクで ls /mnt/logs は動くか（C-5）

いいえ（Stale・bad file handle・応答なし）→ マウント単位：仮説 1（FS の作り直し）・原因 5〜7。CloudTrail（C-7）で作り直しの時刻を確認し、全タスクを入れ替える。／はい → Q2 へ。

▼

Q2　同じ EFS 上の JBoss の server.log・GC ログ・アクセスログも失敗しているか（C-6）

はい → 共通の親ディレクトリの作り直し・片付け（原因 4・5）。作業記録とジョブを確認。／いいえ → Q3 へ（今回はここ）。

▼

Q3　失敗しているファイルは、複数のタスクが同じパスで使うファイルか（C-8）

はい → 仮説 2（原因 1〜3）を強く疑う。Q4 へ。／いいえ（タスク専用のファイル）→ 片付け・手作業の削除（原因 4）を疑う。

▼

Q4　エラーは毎時 00 分すぎから始まり、正常なタスクが毎時入れ替わるか（C-1・C-4）

はい → 原因 1（ロールオーバの連鎖）で確定に近い。Q5 で最終確認。／いいえ（特定の時刻だけ）→ 片付けやデプロイの時刻（原因 2・4）と突き合わせる。

▼

Q5　FD の判定が「CURRENT 1・残りは STALE（ARCHIVE 0〜1）」か（C-2）。trace.log.* が 1 タスク分か（C-3）

はい → 原因 1 で確定。10_緊急対応 へ。／いいえ → ほかの書き手（原因 3）を C-8・C-9 で探す。

▼

Q6　デプロイの直後に増えるか（C-7 のサービスイベント）

はい → 原因 2 も重なっている。対処は原因 1 と同じ（書き手を分ける）＋デプロイの時刻をずらす。

■ 観察と原因の対応表（見分け方の早見）

観察	共有ファイルの連鎖（1・2）	ほかの書き手（3）	片付け（4）	ディレクトリの作り直し（5）	FS の作り直し（6・7）
ls /mnt/logs が Stale／応答なし	×	×	×	△（消した階層による）	○
JBoss のログも失敗	×	×	△	△	○

共有ファイルだけ失敗	○	○	△	△	×
毎時 00 分すぎに始まる	○	○	×	×	×
正常なタスクが毎時入れ替わる	○	○	×	×	×
CURRENT 1・残り STALE	○	○	×	×	×（全部 MOUNTSTALE）
再起動で直り、二度と起きない	×（次の 00 分にまた）	×	○	○	○
CloudTrail に EFS の削除・作成	×	×	×	×	○

○ = その原因ならこう見える、× = 見えない、△ = 場合による。今回の観察（JBoss のログは無事・tracelog だけ・一部のタスクだけ）は「共有ファイルの連鎖」の列とだけ一致する。

10　緊急対応：今すぐ止める（再起動では直らない）

再起動（タスクの入れ替え）は、全タスクが同じ tracelog を開き直すだけなので、次の 00 分にまた連鎖する。止めるには「書き先を分ける」設定のデプロイが要る。

■ なぜ再起動では直らないのか　新しい 6 タスクは起動時に同じ tracelog を開く（全員 CURRENT に見える）。しかし次の 00 分に 6 回のロールオーバが起き、また 1 タスクだけが残る。それどころか、起動したタスクから見て tracelog の作成時刻が前の時間帯だと、最初のログで即ロールオーバして、その場で連鎖を起こす。

1　確定する（10〜20 分）

08 の C-1（server.log の時刻）、C-2（6 タスクの FD 判定）、C-3（trace.log.* の行数と更新時刻）、C-5（ls /mnt/logs）を行う。「マウントは正常・CURRENT 1・残り STALE・保存ファイルが 1 タスク分」なら原因 1 で確定。

▼

2　影響を記録する

いつから、どのタスクの、どの時間帯のトレースログが失われたかを記録する（監査・障害報告のため）。失われたログは EFS 上から消えているので取り戻せない。EFS のバックアップ（AWS Backup）があっても、1 時間の中で消えた行は入っていないことが多い。

▼

3　書き先をタスクごとに分ける設定を作る

下の表の案 A〜C から、自分たちのパイプラインで最も早く出せるものを選ぶ。どれも「1 つのファイルに書くのは 1 タスクだけ」になる。あわせてレイアウトに時刻とタスク名を入れると、以後の調査が楽になる（11 の設定例）。

▼

4　デプロイする（時刻に注意）

毎時 05 分〜50 分のあいだに始めて終える。新旧タスクが重なっている間に 00 分をまたぐと、古いタスクどうしの連鎖がもう一度起きる（新しいタスクは影響を受けない）。

▼

5　確かめる

次の 00 分を過ぎてから 08 の C-2 を全タスクで実行し、全タスクが CURRENT（それぞれ自分のディレクトリ・ファイル）であることを確認する。server.log に「Stale file handle」「Unable to close stream」が出ないことを 2〜3 時間見る。

▼

6　古い共有ファイルを片付ける

全タスクが新しい書き先に移ったあとで、共有ディレクトリの tracelog・trace.log.* を保管場所へ移す（動いているタスクが使っていないことを C-2 で確認してから）。

■ 書き先を分ける 3 つの案

案	やること	変えるもの	向いている場合	注意
A：-Diwin.log を JBoss と同じ mid/{タスクID} に（おすすめ）	JBoss のログ用に既にタスク ID を求めている起動スクリプトで、-Diwin.log=/mnt/logs/front/logs/inter-api/mid/<タスクID> も渡す（バックも同様）。Log4j2 は起動時にディレクトリを作るが、念のため mkdir -p してもよい	起動スクリプト（イメージ）・タスク定義の環境変数	既に動いている mid/{タスクID} の仕組みをそのまま使える。log4j2.xml を変えずに済む。readonlyRootFilesystem=true でも使える（再現実験 C3 で確認）	-Diwin.log の指定場所（standalone.conf・JAVA_OPTS・standalone.xml の system-properties）を確認し、後から上書きされないようにする。mid/{タスクID} の片付けのルールにトレースログも含める
B：リンク先をタスクごとに	起動スクリプトで /webapp/webapp9mf02/logs のリンクを /mnt/logs/front/logs/inter-api/mid/<タスクID> に張り替える（ln -sfn）	起動スクリプト（イメージ）	-Diwin.log を変えにくい場合	ルートファイルシステムが読み取り専用だと張り替えられない → 案 A
C：log4j2.xml でタスクごとの名前に	fileName="${sys:iwin.log}/${hostName}/tracelog"、filePattern="${sys:iwin.log}/${hostName}/trace.log.%d{yyyy-MM-dd-HH}" にする	log4j2.xml（WAR の中ならアプリの再ビルド）	起動スクリプトを触れない場合	${hostName} は Log4j2 が用意する値（LoggerContext.java 691 行）。Fargate ではタスクの IP から作られた名前なので、同時に動くタスクでは重ならない。後で同じ IP のタスクが来ると同じディレクトリに追記される

■ デプロイできないときの一時しのぎ（効果は限定的）

方法	効果	副作用・注意
何もしない（監視だけ強める）	なし。毎時 5/6 ほどのトレースログが失われ続ける	影響を記録し、早くデプロイする
タスク数を 1 にする	書き手が 1 つになり、連鎖は止まる	処理能力が 1/6。本番では現実的でない
監視の間隔を短くし、失敗の多いタスクを手で入れ替える	ほぼ無し（入れ替えても次の 00 分にまた起きる）	おすすめしない
log4j2.xml が外部ファイルで monitorInterval が設定されている場合、その場で書き換える	設定の再読み込みで書き先を変えられる	各タスクのコンテナ内のファイルなら全タスクで書き換えが要る。イメージに焼いたファイルは入れ替えで元に戻る

11　恒久対策：1 ファイル 1 書き手・正本を CloudWatch Logs に

原則は「1 つのファイルに書くのは 1 つの JVM だけ」。JBoss のログ（mid/{タスクID}）で既にできていることを、トレースログにも当てはめる。

■ 原則　Log4j2 のロールオーバは、同じ JVM の中でしか調整されない。だから「書き手が 1 つ」なら、EFS の上でも安全に動く（再現実験 C3：6 タスク・Stale 0・欠けた行 0）。逆に、書き手が 2 つ以上なら、設定をどう工夫しても（locking を含めて）連鎖は止まらない。

■ 11-1　対策の一覧（優先順）

優先	対策	効果	手間	注意
必須	P1 トレースログの書き先をタスクごとに（-Diwin.log を mid/{タスクID} に。10 の案 A〜C）	連鎖が起きなくなる	小（起動スクリプト）	フロントもバックも。デプロイは 00 分の前後を避ける
必須	P2 片付けは止まったタスクのディレクトリだけ（ECS の一覧と照合）	動いているタスクのファイルを消さない	小	P1 でディレクトリが増えるのでセットで
強く推奨	P3 レイアウトに時刻・タスク・スレッドを入れる（PatternLayout）	どのタスクの行か後から追える	小（log4j2.xml）	1 行が長くなる。容量を見積もる
推奨	P4 トレースログの正本を CloudWatch Logs に（サイドカー／FireLens／awslogs）	EFS の事故・タスクの入れ替えと無関係にログが残る	中	料金（取り込み・保存）と保存期間を決める
推奨	P5 Failover の保険（A4 が書けないときは標準出力へ逃がす）	ファイルに書けない間のログも残る	小（log4j2.xml）	A4 に ignoreExceptions="false" が要る（Failover の決まり）
推奨	P6 監視（メトリクスフィルタ＋アラーム）	起きたらすぐ気づく	小	Log4j2 の間引きがあるので「1 件以上」で鳴らす
推奨	P7 ヘルスチェックの設計（13）	誤った一斉入れ替えを防ぎつつ、状態を見える化	中（Spring MVC）	根本対策の前に ELB を 503 にしない
推奨	P8 Log4j を 2.25.5 以降（または 2.26.1）へ	2026 年の CVE への対応（本件とは無関係）	小	log4j-api／core／slf4j2-impl をそろえて上げる
推奨	P9 パスの命名規則（環境/サービス/コンテナ/タスク ID）を文書にし、レビューで確認	同じ事故の再発防止	小	テスト環境も同じ規則に

■ 11-2　起動スクリプトの例（案 A：-Diwin.log を mid/{タスクID} に）

JBoss のログ用に既にタスク ID を求めているなら、その値をそのまま使う。下は、ECS のタスクメタデータ（v4）からタスク ID を求める一般的な例。

entrypoint の一部（bash。UBI 9 の curl･sed･grep だけで動く）

```
#!/bin/bash
set -eu
BASE=/mnt/logs/front/logs/inter-api              # バックは /mnt/logs/back/logs/inter-api
TASK_ID=""
if [ -n "${ECS_CONTAINER_METADATA_URI_V4:-}" ]; then
  ARN="$(curl -fsS --max-time 2 "${ECS_CONTAINER_METADATA_URI_V4}/task" | grep -o '"TaskARN":"[^"]*"' | head -n 1 | cut -d'"' -f4)" || ARN=""
  TASK_ID="${ARN##*/}"
fi
[ -n "${TASK_ID}" ] || TASK_ID="$(uname -n)-$(date +%Y%m%d%H%M%S)"   # 取れないときの予備（重ならない名前）
LOGDIR="${BASE}/mid/${TASK_ID}"                    # JBoss･GC･アクセスログと同じ場所
mkdir -p "${LOGDIR}"
# JBoss の起動コマンドに -D で渡す（standalone.sh は -D をシステムプロパティとして JVM に渡す）。
# JAVA_OPTS を丸ごと export すると、standalone.conf の既定値（ヒープなど）が使われなくなるので注意。
exec <EAP_HOME>/bin/standalone.sh -b 0.0.0.0 -Diwin.log="${LOGDIR}" "$@"
```

･-Diwin.log が standalone.xml の system-properties や standalone.conf で固定されていると、起動スクリプトの値が上書きされる。どこで決めているかを先に確かめる（08 の C-8）。

･ログのファイル名は今のまま（tracelog･trace.log.%d{yyyy-MM-dd-HH}）でよい。ディレクトリが分かれれば、ロールオーバは各タスクの中で 1 回だけになる。

･タスク ID のディレクトリは、タスクが止まった後も残る。片付け（11-5）とセットで入れる。

■ 11-3 log4j2.xml の例（タスクごと＋レイアウト＋Failover の保険）

案 C（${hostName}）とレイアウト･Failover をまとめた例。案 A なら fileName／filePattern は今のままでよい

```
<Configuration>
  <Appenders>
    <RollingFile name="A4" ignoreExceptions="false"
                 fileName="${sys:iwin.log}/${hostName}/tracelog"
                 filePattern="${sys:iwin.log}/${hostName}/trace.log.%d{yyyy-MM-dd-HH}">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss.SSS} ${hostName} [%t] %m%n"/>
      <Policies><TimeBasedTriggeringPolicy/></Policies>
    </RollingFile>
    <Console name="TRACE_STDOUT" target="SYSTEM_OUT">
      <PatternLayout pattern="TRACE-FAILOVER %d{yyyy-MM-dd HH:mm:ss.SSS} ${hostName} [%t] %m%n"/>
    </Console>
    <Failover name="A4F" primary="A4" retryIntervalSeconds="60">
      <Failovers><AppenderRef ref="TRACE_STDOUT"/></Failovers>
    </Failover>
```

```
  </Appenders>
  <Loggers>
    <Logger name="jp.iwin.base.tango.ILGLog4j.trace" level="debug" additivity="false">
      <AppenderRef ref="A4F"/>
    </Logger>
  </Loggers>
</Configuration>
```

・Failover は「主のアペンダ（A4）が例外を投げたら、代わりのアペンダへ」という仕組み。そのため A4 に ignoreExceptions="false" が要る（Log4j2 公式）。Failover 自身は既定で例外を外に出さないので、アプリ（Spring MVC）には届かない。

・代わりの先は標準出力。JBoss の設定によっては標準出力が server.log（[stdout]）にも入る。CloudWatch Logs に送っているなら、そこで拾える。

・retryIntervalSeconds（既定 60 秒）ごとに A4 を試し直す。Stale のままなら、次のロールオーバまで標準出力へ逃げ続ける。

・書き先を分けた後は Stale はほぼ起きないので、Failover は「もしもの保険」。監査でトレースログの欠けが許されない場合に入れる。

■ 11-4 ログの正本を CloudWatch Logs に

方法	しくみ	よいところ	気をつけるところ
サイドカー（CloudWatch Agent／Fluent Bit）	同じタスクに収集用のコンテナを置き、自分のタスクの mid/{タスクID} のファイルを読んで送る	今のファイル出力を変えずに足せる。タスクごとのログストリームにできる	サイドカーにも同じタスク ID のディレクトリを教える（起動時に同じ方法で求める）
FireLens（awsfirelens ログドライバ）	アプリが標準出力に出したログを Fluent Bit が受けて送る	ファイルを使わない（EFS と無関係）	JBoss の標準出力の形（[stdout] の前置き）を整える必要がある
awslogs ログドライバ	コンテナの標準出力をそのまま CloudWatch Logs へ	いちばん簡単	標準出力に出すログの量・形を決める必要がある

EFS は「ログを一時的に置く場所」、CloudWatch Logs は「正本」と役割を分けると、EFS の作り直し・片付けの事故でログを失わなくなる。既にサイドカーで JBoss のログを送っているなら、トレースログも同じ仕組みに乗せるのがいちばん早い。

■ 11-5 片付け（ハウスキーピング）のルール

・消してよいのは「止まったタスク」のディレクトリ（mid/{タスクID}）だけ。動いているタスクの一覧（aws ecs list-tasks）と照合し、含まれるものは消さない。

・日数だけで消さない。動いているタスクでも、しばらく書かれないファイルはある。

・共有ディレクトリ（inter-api の直下）の古い trace.log.* は、全タスクが新しい書き先に移ったことを 08 の C-2 で確認してから移す。

・片付けの記録（いつ・何を・誰が）を残す。CloudTrail には EFS の中のファイル操作は残らない。

動いているタスクのディレクトリを除いて、7 日より古い mid/{タスクID} を一覧する（消す前に必ず目で確認）

```
RUNNING=$(aws ecs list-tasks --cluster <クラスタ> --service-name <サービス> --desired-status RUNNING --query 'taskArns[]' --output text | tr '¥t' '¥n' | sed 's#.*/##')

find /mnt/logs/front/logs/inter-api/mid -mindepth 1 -maxdepth 1 -type d -mtime +7 | while read -r d; do
  id="${d##*/}"; echo "${RUNNING}" | grep -qx "${id}" && continue; echo "消してよい候補: ${d}"
done
```

■ 11-6 監視（メトリクスフィルタとアラーム）

server.log を CloudWatch Logs に送っている場合（ロググループ名は環境に合わせる）

```
aws logs put-metric-filter --log-group-name <server.log のロググループ> --filter-name tracelog-write-error ¥
  --filter-pattern '?"Stale file handle" ?"Unable to write to stream" ?"Unable to close stream"' ¥
  --metric-transformations metricName=TraceLogWriteError,metricNamespace=App/Logging,metricValue=1,defaultValue=0
aws cloudwatch put-metric-alarm --alarm-name tracelog-write-error --namespace App/Logging --metric-name TraceLogWriteError ¥
  --statistic Sum --period 300 --evaluation-periods 1 --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold ¥
  --treat-missing-data notBreaching --alarm-actions <SNS トピックの ARN>
```

Log4j2 は同じアペンダのエラーを「最初の数件のあとは 5 分に 1 件」に間引くので、5 分の合計が 1 件以上で鳴らすのがちょうどよい。「Unable to close stream」は間引かれないので、毎時 00 分すぎに並べば共有ファイルの連鎖の目印になる。

■ 11-7 Log4j の版（2026 年 10 月時点）

版	公開日	主な内容	今回の設定への影響
2.25.3（今の版）	2025-12-15	Socket アペンダの TLS ホスト名確認の修正（CVE-2025-68161）	―
2.25.4	2026-03-28	CVE-2026-34477（<Ssl> の verifyHostName が無視される）・34478（Rfc5424Layout）・34480（XmlLayout）の修正	RollingFile＋既定の PatternLayout だけなら当たらない見込み。他の設定ファイルも確認
2.26.0	2026-05-02	機能の追加・整理	―
2.26.1／2.25.5	2026-06-29／2026-07-01	log4j-api の CVE-2026-49844（MapMessage.asJson の NaN・Infinity）の修正など	JsonTemplateLayout などで MapMessage を使っていなければ当たらない見込み

どの版にも、今回の「複数の JVM が同じファイルをロールオーバする」問題を直す変更は無い（Log4j2 は 1 つの JVM の中での調整しかしない設計）。版を上げても直らないので、P1 が先。

12　運用回避：やってはいけない操作・作り直しの手順・チェックリスト

根本対策が入るまでの間、そして入った後も守る運用のルール。EFS の上では「動いているタスクの足元を動かさない」が合言葉。

■ 12-1　やってはいけない操作

操作	何が起きるか	代わりにどうするか
タスクが動いたまま EFS のファイルシステム・マウントターゲット・アクセスポイントを作り直す	全タスクのマウント全体が Stale・応答なし（仮説 1）	12-2 の手順で 0 タスクにしてから
動いているタスクが使うディレクトリを rm -rf して作り直す	そのディレクトリのファイルを開いていた全タスクが Stale	作り直さない。起動スクリプトは mkdir -p だけ
動いているタスクのファイルを手で消す・移す・上書きする	そのタスクが Stale（または別のファイルに書く）	止まったタスクのものだけを片付ける
共有ファイルのまま、毎時 00 分の前後にデプロイ・スケールアウト	新旧の書き手が増え、連鎖が長くなる	05 分〜50 分の間に始めて終える
同じ EFS・同じパスをテスト環境と本番で共用	別環境のタスクが書き手になる	環境ごとに EFS かアクセスポイント（rootDirectory）を分ける
根本対策の前に、ヘルスチェックをトレースログの状態で 503 にする	毎時 4〜5 タスクが不健康 → 一斉入れ替え	13 の段階に沿って進める

■ 12-2　EFS（やディレクトリ）を作り直すときの手順

1　影響範囲を決める

そのファイルシステム（アクセスポイント）を使うサービス・タスク定義・環境をすべて洗い出す（ClientConnections も見る）。

▼

2　書き手を止める

対象サービスの desiredCount を 0 にし、タスクが 0 になったことを確かめる（デプロイ中のタスクにも注意）。

▼

3　必要なログを退避する

EFS 上のログを別の場所へコピーする（作り直すと消える）。

▼

4　作り直す

FS／マウントターゲット／アクセスポイントを作り直す。IaC なら plan の差分を確認してから。

▼

5　タスク定義を合わせる

新しい fileSystemId・accessPointId をタスク定義に入れ、新しいリビジョンにする（古い FS を指したままだと起動に失敗する）。

▼

6　起動して確かめる

desiredCount を戻し、各タスクで ls /mnt/logs・08 の C-2 を実行する。

■ 12-3　根本対策が入るまでの約束（共有ファイルのままの間）

・トレースログは毎時およそ 5/6 が失われている前提で、障害調査は JBoss の server.log・アクセスログ・アプリの別のログで補う。

・障害の時刻が分かっている場合は、その時間の CURRENT のタスク（08 の C-1・C-2 で特定）のログだけが残っている、と考える。

・デプロイ・スケールの時刻を毎時 05 分～50 分に寄せる。

・トレースログの欠けを、監査・報告の窓口に早めに知らせる。

■ 12-4　チェックリスト（変更の前後で確認する）

済	確認すること	どうやって
	フロント・バックとも、tracelog の書き先がタスクごとに分かれている	08 の C-8（-Diwin.log・リンク先）
	全タスクで tracelog-fd-check.sh が CURRENT	08 の C-2（毎時 02 分以降）
	server.log に Stale file handle・Unable to close stream が出ていない（2～3 時間）	08 の C-1・11-6 のアラーム
	ls /mnt/logs と mid/{タスクID} の更新が正常	08 の C-5・C-6
	片付けは止まったタスクのディレクトリだけを対象にしている	11-5
	EFS を共用している別環境・別サービスが無い（ClientConnections が想定どおり）	08 の C-9
	ELB のヘルスチェックはリクエストの処理能力だけを見ている（503 の条件を決めた）	13
	Log4j を 2.25.5 以降（2.26.1）にした、または計画に入れた	11-7

「済」の欄は ○（済）／×（未）／－（対象外）から選ぶ。

13　ヘルスチェック：Spring MVC の API が 200 を返す理由と、安全な設計

結論：200 は Controller の握りつぶしではなく、Log4j2 が例外をアプリに渡さない設計（ignoreExceptions=true）の結果。ヘルスチェックで気づきたいなら、例外を待たずに「トレースログの状態を自分で調べる」必要がある。ただし今の構成のまま ELB を 503 にしてはいけない。

■ 13-1　いまのヘルスチェックでわかること・わからないこと

項目	わかるか	理由
JBoss が動き、Spring MVC の DispatcherServlet が応答できる	わかる	HTTP で 200 が返ってくること自体が証拠
業務の処理（DB など）ができる	API が調べていれば	調べていなければわからない（今は常に 200）
トレースログが書けている	わからない	Log4j2 は書けなくても例外をアプリに渡さない。API も調べていない
EFS のマウントが生きている	ほぼわからない	API がファイルに触れなければ、マウントが止まっていても 200（触れると応答が止まり、タイムアウトで失敗に見える）

■ 13-2　200 が返る理由：3 つの可能性の見分け

可能性	あてはまるか	根拠・確かめ方
① Log4j2 が例外を受け止めているので、Controller に届いていない（設計どおり）	あてはまる（今回の主な理由）	A4 の ignoreExceptions は既定の true。AppenderControl.java 168〜173 行で処理が終わり、log4j-api の AbstractLogger.java 2962〜2988 行も呼び出し元へ投げない。再現実験でもアプリは 1 件も例外を受け取らずに動き続けた
② Controller・@ControllerAdvice・Filter が例外を握りつぶして 200 にしている（実装ミス）	今回の Stale については、あっても関係しない（届く例外が無い）。別の異常を隠していないかは確認する価値あり	13-4 の観点でコードを見る（catch (Exception) で 200、@ExceptionHandler(Exception.class) で 200 など）
③ ヘルスチェックが何も調べていない（常に 200 を返す作り）	あてはまる（①と組み合わさって 200 になる）	「Stale を検知できない」のは、検知する処理が無いため。設計上の不足であって、例外処理のミスではない

■「意図的に catch してスローし直さないと検知できないのか」への答え　catch しようにも、例外は Controller まで来ない。Spring MVC 側で何を書いても、既定の Log4j2 からは何も受け取れない。ignoreExceptions="false" にすれば AppenderLoggingException が logger.debug() から飛んでくるが、それは「ログを書いた業務のリクエスト」が失敗するだけで、ヘルスチェック API には届かない（ヘルスチェック API がトレースログを書かない限り）。ヘルスチェックで見たいなら、ヘルスチェック自身が状態を調べる。

■ 13-3　Spring MVC で、例外が HTTP の状態になるまで

1　Controller のメソッドが例外を投げる

例：業務処理の中で RuntimeException。Log4j2 の Stale は（既定では）ここまで来ない。

▼

2　DispatcherServlet が HandlerExceptionResolver に順に聞く

@ExceptionHandler（@ControllerAdvice を含む）→ @ResponseStatus・ResponseStatusException → Spring MVC の標準の例外（405・415 など）の順。

▼

3　どれも処理しなければ、サーブレットコンテナへ

例外は DispatcherServlet から外へ出て、JBoss（Undertow）が 500 のエラーページを返す。Spring MVC の既定では 503 にはならない。

▼

4　503 にしたいとき

ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)、throw new ResponseStatusException(HttpStatus.SERVICE_UNAVAILABLE)、例外クラスに @ResponseStatus(HttpStatus.SERVICE_UNAVAILABLE)、@ExceptionHandler で 503 を返す、のどれかで明示する。（Spring Boot Actuator の health は、DOWN を 503 にする既定を持つ。Boot を使っていなければ自分で決める）

■ 13-4　握りつぶしがないか、コードで確かめる観点

見る場所	握りつぶしの例	なぜ問題か
ヘルスチェックの Controller	try { … } catch (Exception e) { return ResponseEntity.ok("OK"); }	調べた結果の異常まで 200 にしてしまう
@ControllerAdvice／@ExceptionHandler	@ExceptionHandler(Exception.class) で 200 と本文だけを返す	すべての API の失敗が 200 に見える（ELB も監視も気づけない）
自作の HandlerExceptionResolver	response.setStatus(200) にしてしまう	同上
Servlet Filter（OncePerRequestFilter など）	chain.doFilter を catch で包み、例外を飲み込む	Spring MVC の外で消えるので気づきにくい
web.xml の error-page と、その先のページ	エラーページを返す Controller が 200 で返す	500 のはずが 200 に変わる
ログ呼び出しの周り	try { log.debug(…) } catch (Exception e) { }	既定の Log4j2 では投げられないので今は無害。ignoreExceptions="false" にしたときに効いてくる

■ 13-5　なぜ今の構成のまま 503 にしてはいけないのか

時刻	タスク（6 つ）	ELB（30 秒ごと・2 回失敗で外す・5 回成功で戻す）	ECS サービス

13:00:01	**連鎖で 5 タスクが STALE。ヘルスチェックが 503 を返し始める**		
13:00:30〜13:01:00		**2 回続けて失敗した 5 タスクを外す。1 タスクに全リクエストが集まる**	
13:01〜13:03	**残った 1 タスクが過負荷になる**	**（全部が不健康なら、全部に振り分ける fail-open）**	**ELB で不健康なタスクを止め、新しいタスクを起動**
13:03〜13:06	新しいタスクは今の tracelog を開く → CURRENT に見える	新しいタスクは最初のヘルスチェック 1 回の成功で振り分けに入る（外したタスクが戻るには 5 回連続の成功＝約 2.5 分）	
14:00:01	**また連鎖。新しいタスクも含めて 5 タスクが STALE**	**また外す**	**また入れ替える（毎時くり返す）**

入れ替えても共有ファイルは変わらないので、毎時同じことがくり返され、そのたびに処理能力が落ちる。だから順番は「根本対策 → 監視 → 必要ならヘルスチェックに含める」。

■ 13-6 安全な設計（段階）

段階	ELB のヘルスチェック	トレースログの見方	条件
0（今すぐ）	そのまま（200）。リクエストを処理できるかだけを見る	server.log のメトリクスフィルタ＋アラーム（11-6）	—
1（根本対策と同時）	そのまま	トレースログの状態を返す専用の API（/health/tracelog）を追加し、監視・ECS Exec で見る	13-7 の実装例
2（根本対策の後・必要なら）	STALE が数分続いたら 503 にする、などゆとりを持たせて含めるかを判断	専用の API の結果を使う	1 ファイル 1 書き手になっていること。全タスク同時の失敗は fail-open になる点も考える

■ 13-7 Spring MVC の実装例：トレースログの状態を調べる API

例外を待つのではなく、Java のプロセスが開いているファイル（/proc/self/fd）を自分で調べる。tools/tracelog-fd-check.sh と同じ考え方。Linux（Fargate）専用。EAP 8.1 の Spring MVC（jakarta.*）を想定。

TraceLogProbe.java（状態を調べる部品）

```
import java.io.IOException;
import java.nio.file.*;

/** -Diwin.log のディレクトリ直下で、このプロセスが開いているトレースログの状態を返す */
public final class TraceLogProbe {
```

```
    public enum State { CURRENT, NOT_CURRENT, STALE, NOFD, MOUNTSTALE, ERROR }

    public static State check() {
        Path dir = Paths.get(System.getProperty("iwin.log", "/webapp/webapp9mf02/logs"));
        try {
            Path real = dir.toRealPath();                       // リンクをたどる
            Object cur = Files.getAttribute(real.resolve("tracelog"), "unix:ino");
            try (DirectoryStream<Path> fds = Files.newDirectoryStream(Paths.get("/proc/self/fd"))) {
                for (Path fd : fds) {
                    Path target;
                    try { target = Files.readSymbolicLink(fd); } catch (IOException e) { continue; }
                    Path p = Paths.get(target.toString().replace(" (deleted)", ""));
                    if (!real.equals(p.getParent())) continue;          // そのディレクトリの直下だけ
                    String n = p.getFileName().toString();              // トレースログだけ（server.log などは数えない）
                    if (!n.equals("tracelog") && !n.startsWith("trace.log.")) continue;
                    try {
                        Object ino = Files.getAttribute(fd, "unix:ino");    // リンクをたどって stat → EFS に問い合わせ
                        return ino.equals(cur) ? State.CURRENT : State.NOT_CURRENT;
                    } catch (FileSystemException e) {
                        return String.valueOf(e.getReason()).contains("Stale file handle") ? State.STALE : State.ERROR;
                    }
                }
            }
            return State.NOFD;
        } catch (IOException e) {
            return String.valueOf(e.getMessage()).contains("Stale file handle") ? State.MOUNTSTALE : State.ERROR;
        }
    }
}
```

TraceLogHealthController.java（専用の API。段階 1 では監視からだけ呼ぶ）

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class TraceLogHealthController {
    @GetMapping("/health/tracelog")
    public ResponseEntity<String> tracelog() {
```

```
        TraceLogProbe.State s = TraceLogProbe.check();
        HttpStatus st = (s == TraceLogProbe.State.CURRENT) ? HttpStatus.OK : HttpStatus.SERVICE_UNAVAILABLE;
        return ResponseEntity.status(st).body("tracelog=" + s);     // 例外ではなく、調べた結果で 503 を返す
    }
}
```

・ELB のヘルスチェックのパス（今の API）は変えずに、まずはこの API を監視（外形監視・定期ジョブ・ECS Exec）から呼ぶ。

・ロールオーバの直後（毎時 00 分すぎの 1 分ほど）は、NFS の名前のキャッシュで NOT_CURRENT を CURRENT と見ることがある。STALE は EFS の返事そのものなので確か。

・1 回の呼び出しで EFS への問い合わせは数回（stat）。30 秒ごとでも負担は小さい。ただし EFS が応答しないときに固まらないよう、監視側でタイムアウトを短く。

・別の方法として、A4 に自作の ErrorHandler を付けて最後の失敗時刻を覚えさせ、API で返すこともできる（Log4j2 の内部に依存するので、上の方法のほうが単純）。

■ 13-8 ECS のコンテナヘルスチェックとの違い

	ELB（ALB）のヘルスチェック	ECS のコンテナヘルスチェック
だれが確かめる	ロードバランサがネットワーク越しに HTTP で	コンテナの中でコマンドを実行（終了コード 0 が成功）
既定の間隔など	30 秒・タイムアウト 5 秒・2 回失敗で外す・5 回成功で戻す・成功コード 200	タスク定義で決める（interval・timeout・retries・startPeriod）
失敗すると	振り分けから外す。ECS サービスは不健康と報告されたタスクを入れ替える	必須（essential）コンテナが UNHEALTHY ならタスクが UNHEALTHY → サービスが入れ替える
今回の使い方	リクエストを処理できるかだけを見る	トレースログの状態は入れない（13-5 の理由）

14　コマンド集：調査・確認に使うものを 1 か所に

<クラスタ>・<サービス>・<コンテナ> は環境に合わせる。読むだけのコマンドから順に並べた。ECS Exec には Session Manager プラグインが要る。

■ 14-1 タスクの一覧と ECS Exec

bash（CloudShell・Linux・Mac）

```
aws ecs list-tasks --cluster <クラスタ> --service-name <サービス> --desired-status RUNNING --query 'taskArns[]' --output text
aws ecs execute-command --cluster <クラスタ> --task <タスクID> --container <フロントのコンテナ名> --interactive --command "/bin/bash"
```

PowerShell（Windows）

```
$Cluster = "<クラスタ>"; $Service = "<サービス>"; $Container = "<フロントのコンテナ名>"

$tasks = (aws ecs list-tasks --cluster $Cluster --service-name $Service --desired-status RUNNING --query "taskArns[]" --output text) -split "¥s+" | Where-Object { $_ }

foreach ($t in $tasks) { Write-Host "=== $($t.Split('/')[-1])"; aws ecs execute-command --cluster $Cluster --task $t --container $Container --interactive --command "/bin/bash" }
```

■ 14-2 コンテナの中で：トレースログの判定（ツールが無いときの短縮版）

貼り付けて実行（CURRENT・STALE・OTHER を表示。OTHER は ls -i で名前を確かめる）

```
export LC_ALL=C
PID=$(for p in /proc/[0-9]*; do [ "$(readlink $p/exe 2>/dev/null | sed 's#.*/##')" = java ] && echo ${p#/proc/}; done | head -1)

D=$(readlink -f "$(tr '¥0' '¥n' < /proc/$PID/cmdline | sed -n 's/^-Diwin¥.log=//p' | head -1)"); [ -n "$D" ] || D=$(readlink -f /webapp/webapp9mf02/logs)

C=$(stat -c %i "$D/tracelog")

for f in /proc/$PID/fd/*; do t=$(readlink $f); t=${t% (deleted)}; [ "$(dirname "$t")" = "$D" ] || continue; case "${t##*/}" in tracelog|trace.log.*) ;; *) continue;; esac;

  i=$(stat -L -c %i $f 2>&1); case "$i" in *Stale*) echo "STALE fd=${f##*/}";; "$C") echo "CURRENT fd=${f##*/}";; *) echo "OTHER fd=${f##*/} inode=$i";; esac; done

ls -li --time-style=full-iso "$D" | grep -E 'tracelog|trace[.]log[.]'
```

■ 14-3 コンテナの中で：マウント・設定・JBoss のログ

```
ls /mnt/logs >/dev/null && echo MOUNT_OK
grep ' /mnt/logs ' /proc/mounts
readlink -f /webapp/webapp9mf02/logs /webapp/webapp9mb02/logs 2>/dev/null
tr '¥0' '¥n' < /proc/$PID/cmdline | grep -E 'iwin.log|jboss.server.log.dir|Xlog:gc|log4j'
```

```
ls -l --time-style=full-iso /mnt/logs/front/logs/inter-api/mid/ | tail -n 8
grep -n -E 'Stale file handle|Unable to write to stream|Unable to close stream' <server.log のパス> | tail -n 20
```

■ 14-4 CloudWatch Logs Insights（server.log を送っている場合）

時間ごと・ログストリーム（タスク）ごとのエラーの数

```
fields @timestamp, @logStream, @message
| filter @message like /Stale file handle|Unable to write to stream|Unable to close stream/
| stats count(*) as n by @logStream, bin(1h)
| sort bin(1h) desc
```

毎時 00 分すぎに「Unable to close stream」が並ぶか（連鎖の目印）

```
fields @timestamp, @logStream
| filter @message like /Unable to close stream/
| stats count(*) as n, min(@timestamp) as first by bin(1h)
```

■ 14-5 AWS の記録

CloudTrail（EFS の作成・削除）・ECS（デプロイの履歴）・EFS の接続数

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventSource,AttributeValue=elasticfilesystem.amazonaws.com ¥
  --start-time 2026-10-01T00:00:00Z --query 'Events[].[EventTime,EventName,Username]' --output table

aws ecs describe-services --cluster <クラスタ> --services <サービス> --query 'services[0].deployments[].[createdAt,status,taskDefinition,runningCount]' --output table

aws ecs describe-services --cluster <クラスタ> --services <サービス> --query 'services[0].events[0:30].[createdAt,message]' --output table
aws cloudwatch get-metric-statistics --namespace AWS/EFS --metric-name ClientConnections --dimensions Name=FileSystemId,Value=<fs-ID> ¥
  --statistics Sum --period 60 --start-time 2026-10-10T03:00:00Z --end-time 2026-10-10T04:00:00Z
```

■ 14-6 ELB のヘルスチェックの状態

```
aws elbv2 describe-target-health --target-group-arn <ターゲットグループの ARN> --query 'TargetHealthDescriptions[].[Target.Id,TargetHealth.State,TargetHealth.Reason]' --output table

aws elbv2 describe-target-groups --target-group-arns <ターゲットグループの ARN> --query
'TargetGroups[0].[HealthCheckPath,HealthCheckIntervalSeconds,HealthCheckTimeoutSeconds,HealthyThresholdCount,UnhealthyThresholdCount,Matcher.HttpCode]'
```

■ 14-7 再現実験（このフォルダの lab/）

Docker（--privileged が使えること）があれば、約 20 分で 4 ケースを再現する

```
bash lab/run_lab6.sh              # C1〜C4 すべて
bash lab/run_lab6.sh C1 C3        # 指定ケースだけ
python -X utf8 lab/analyze.py lab/results/C1   # 集計（analysis.md・analysis.json）
```

```
python -X utf8 build/build_docs.py          # この資料（Excel・Markdown）を作り直す
```

15　検証結果：Docker で 6 タスク＋NFS を立てて再現した（Log4j 2.25.3）

EFS の代わりに Linux の NFS サーバ（knfsd）、ECS タスクの代わりに UBI 9.8＋OpenJDK 21＋Log4j 2.25.3 のコンテナ 6 台。ロールオーバの周期だけ 1 時間 → 1 分に縮めた。3 回目は本番と同じディレクトリ配置で行い、本文の数字はこれを使った。

■ 15-1　環境

項目	内容
日時（日本時間）	2026-10-10。1 回目 17:04〜、2 回目 18:35〜（どちらも旧い配置）、3 回目 18:49〜（本番と同じ配置。C3 だけ 19:08 に診断ツールを直して取り直し）
Docker・カーネル	29.8.1 kernel=5.15.153.1-microsoft-standard-WSL2
クライアント（タスクの代わり）	Red Hat Enterprise Linux release 9.8 (Plow) java=openjdk version "21.0.12.1" 2026-08-18 LTS log4j=2.25.3
サーバ（EFS の代わり）	Debian 12.15 knfsd（Docker の VM のカーネル）
タスク数・書く間隔	6 タスク。各タスクが 100〜400 ミリ秒ごとに 1 行（リクエストの代わり）
Log4j2 の設定	A4 はユーザーの設定そのまま（RollingFile・TimeBasedTriggeringPolicy・レイアウトなし）。filePattern の最小単位だけ「時」→「分」
配置	/webapp/webapp9mf02/logs → /mnt/logs/front/logs/inter-api（共有）。server.log は /mnt/logs/front/logs/inter-api/mid/T1〜T6（タスクごと）
数え方	各タスクが「何番の行をいつ書いたか」を自分のディスクに記録し、NFS サーバに残った行と突き合わせる（正しい／別の時間帯／消えた）

■ 15-2　ケースと結果の一覧

ケース	条件	結果	わかったこと
C1	いまの構成（6 タスクが同じ tracelog）・NFSv3（EFS と同じく消された実体に Stale を返す）	書いた 5998 行／正しい 17.0%／別の時間帯 0.0%／消えた 83.0%。server.log（mid/{タスク}）は 597/597 行残った	毎分、正しく書けたのは 1 タスク。IOException: Stale file handle を再現。タスクごとの server.log は無事
C2	いまの構成・NFSv4.1（Linux の knfsd：消されても実体を残すサーバ）	書いた 5998 行／正しい 16.7%／別の時間帯 0.0%／消えた 83.3%。Log4j2 のエラー表示 0 件。診断ツールの判定はサーバ側と 24/24 一致	エラーが 1 件も出なくても行は消える。サーバの作りで「エラーの有無」が変わるだけで、原因は同じ。ツール（stat）は STALE を見つけられる
C3	恒久対策（-Diwin.log を mid/{タスク} に。Log4j2 の設定は同じ）・NFSv3	書いた 4572 行／正しい 100.0%／別の時間帯 0.0%／消えた 0.0%。Stale の表示 0 件	1 ファイル 1 書き手にすると、EFS と同じ厳しいサーバでも全部残る

C4	仮説 1（動いている最中に FS を作り直す：中身を消し、同じ IP・別の fsid で出し直す）・NFSv3	作り直しの後：トレース 1367 行中 0 行、server.log 137 行中 0 行が残った。全タスクで ls /mnt/logs が Stale	マウント全体が壊れ、タスクごとの server.log も書けない。今回の観察（JBoss のログは無事）と合わない

■ 15-3 「1 タスクだけ正常」の確認（境目ごとの実際の状態。行の行き先から判定）

回	境目（UTC）	ロールオーバの順番（左が先）	CURRENT	ARCHIVE	STALE	正しく書けたタスク	最後のタスクと
1 回目（旧い配置）	08:05:00	T6 → T3 → T5 → T2 → T1 → T4	1	1	4	T4	一致
1 回目（旧い配置）	08:06:00	T3 → T1 → T4 → T6 → T5 → T2	1	0	5	T2	一致
1 回目（旧い配置）	08:07:00	T6 → T2 → T1 → T5 → T4 → T3	1	0	5	T3	一致
1 回目（旧い配置）	08:08:00	T3 → T6 → T4 → T2 → T5 → T1	1	0	5	T1	一致
2 回目（旧い配置・ツール改良）	09:37:00	T5 → T2 → T4 → T3 → T1 → T6	1	1	4	T6	一致
2 回目（旧い配置・ツール改良）	09:38:00	T4 → T5 → T6 → T1 → T3 → T2	1	0	5	T2	一致
2 回目（旧い配置・ツール改良）	09:39:00	T3 → T1 → T2 → T5 → T4 → T6	1	0	5	T6	一致
2 回目（旧い配置・ツール改良）	09:40:00	T1 → T4 → T5 → T2 → T3 → T6	1	0	5	T6	一致
3 回目（本番と同じ配置）	09:52:00	T1 → T2 → T4 → T5 → T6 → T3	1	0	5	T3	一致
3 回目（本番と同じ配置）	09:53:00	T5 → T6 → T2 → T4 → T1 → T3	1	0	5	T3	一致
3 回目（本番と同じ配置）	09:54:00	T1 → T4 → T2 → T5 → T3 → T6	1	0	5	T6	一致
3 回目（本番と同じ配置）	09:55:00	T5 → T3 → T2 → T1 → T6 → T4	1	0	5	T4	一致

どの回・どの境目でも、正しく書けたのは「その分の最後にロールオーバしたタスク」だった。1・2 回目は、デプロイ直後の最初の境目だけ ARCHIVE が 1 つ出た（05 の型 A が 1 回混ざる）。3 回目は最初から型 B だけだった。

■ 15-4 C1 のタスクごとの数字（3 回目）

タスク	書いた	正しい	別の時間帯	消えた	書き込み失敗の表示	閉じる失敗の表示	server.log 残り/期待
T1	1001	0	0	1001	2	4	100/100
T2	997	0	0	997	2	4	99/99
T3	1007	646	0	361	2	2	100/100
T4	1003	133	0	870	2	3	100/100
T5	998	0	0	998	2	4	99/99
T6	992	241	0	751	2	3	99/99

・「書き込み失敗の表示」は標準エラーに出た Unable to write to stream の数。数百行が消えたタスクでも 2 件ほど（DefaultErrorHandler の間引き）。

・「閉じる失敗の表示」は毎分のロールオーバで出た Unable to close stream: Stale file handle の数（間引かれない）。毎時 00 分すぎに並ぶ目印になる。

・server.log（共有ディレクトリの下の mid/{タスク}）は、同じマウント・同じ親ディレクトリでも 1 行も欠けなかった。

・Java のアプリは 1 件も例外を受け取らずに最後まで動いた（Log4j2 が受け止めている）。

■ 15-5 C1 で各ファイルに残った行（3 回目）

ファイル	どのタスクの行か	どの分（UTC）の行か	壊れ
trace.log.2026-10-10-09-51	T3 158 行	09:51 158 行	NUL 0／壊れた行 0
trace.log.2026-10-10-09-52	T3 241 行	09:52 241 行	NUL 0／壊れた行 0
trace.log.2026-10-10-09-53	T3 247 行	09:53 247 行	NUL 0／壊れた行 0
trace.log.2026-10-10-09-54	T6 241 行	09:54 241 行	NUL 0／壊れた行 0
tracelog	T4 133 行	09:55 133 行	NUL 0／壊れた行 0

保存ファイル（trace.log.*）には、どれも 1 タスク分の行しか残らない。最初の分のファイル（ロールオーバの前に 6 タスクが同じ実体へ追記していた分）も 1 タスク分だけで、これは交換ではなく NFS の追記どうしの上書きによる（04 の 4-8）。名前と中身の分が 1 つずれているファイルがあれば、型 A（上書き）で ARCHIVE のタスクが書いたもの。

■ 15-6 診断ツール（tools/tracelog-fd-check.sh）の正確さ

ケース	サーバ側の実体と一致	ずれた判定

C1 3 回目	24/24	なし
C1 2 回目	23/24	T5：ツール CURRENT／実際 ARCHIVE（snap-183726.txt）
C2	24/24	なし
C3	18/18	なし
C4（作り直しの前後）	12/12	なし

ずれたのは、どれもロールオーバから 30 秒以内（25 秒後）のスナップショットで、ARCHIVE を CURRENT と見たもの（NFS クライアントの名前のキャッシュ）。ツールはこのとき「# 注意: … 秒前に変わったばかり」と出す。毎時 02 分以降に実行すれば避けられる。STALE の判定は一度もずれなかった。

■ 15-7 わかったこと（まとめ）

・6 タスクが同じ tracelog を使うと、毎分（本番では毎時）の境目のあと、正しく書けるのは最後にロールオーバした 1 タスクだけ。

・消えた行は全体の約 8 割。残るのは 1 タスク分（約 1/6）だけで、型 A が起きると名前の時間がずれる。

・ロールオーバの前（全タスクが同じ実体に追記している間）も、NFS の追記の上書きで、残るのはほぼ 1 タスク分だった。エラーは出ない。

・タスクごとのディレクトリ（mid/{タスク}）のログは、同じマウント・同じ親ディレクトリでも無事。違いは書き手の数だけ。

・書き先をタスクごとに分ける（-Diwin.log を mid/{タスク} に）だけで、Stale 0・欠け 0 になった。Log4j2 の設定は変えていない。

・FS の作り直し（仮説 1）は、マウント全体と全タスクのすべてのログを壊す。今回の観察とは見え方がはっきり違う。

・knfsd（NFSv4.1）ではエラーが出ないまま行が消える。エラーが出るかどうかはサーバ次第で、原因（共有ファイル）は同じ。このときも診断ツールは STALE を見つけた（stat はサーバに問い合わせるため）。「エラーが無い＝正常」とは言えない。

■ 15-8 この検証の限界

・サーバは EFS ではなく Linux の knfsd。EFS が「別のクライアントに消された実体に STALE を返す」ことは、EFS 側の技術者の LKML 投稿（2020）と、今回の本番の例外そのもので裏付けた。実験ではその振る舞いを NFSv3 で再現した。

・Fargate のカーネルや EFS のキャッシュの細かな振る舞いは、Docker Desktop（WSL2 のカーネル 5.15）と同じではない。型 A／型 B の割合は環境とリクエストの間隔で変わる。

・ロールオーバの周期を 1 分に縮めた。仕組み（filePattern の最小単位が変わった後の最初のログで交換）は 1 時間でも同じ。

・JBoss・Spring MVC は使っていない（Log4j2 を直接呼ぶ小さな Java プログラム）。Log4j2 の中の動きは同じ。

16　用語集（小学生向けのたとえつき）

むずかしい言葉を、たとえと一緒に。

用語	意味	たとえ
ファイル	名前のついたデータのかたまり	ノート
ディレクトリ	ファイルを入れる入れ物（フォルダ）	たなの段・引き出し
inode（実体）	ファイルの中身そのものと、その番号	ノートそのもの（表紙の名前ではなく）
ファイル名	ディレクトリの中の見出し。付け替えられる	ノートの表紙のラベル
FD（ファイル記述子）	プログラムが開いたファイルを指す番号	手元の番号札
ファイルハンドル	NFS でサーバが発行する、実体を指す引換券	クロークの番号札
ESTALE／Stale file handle	引換券の品物が、もうサーバに無いというエラー（116）	「その番号のノートはもうありません」
シンボリックリンク	別の場所を指す道しるべのファイル	「ノートは職員室のたなにあります」という貼り紙
NFS	ネットワークの向こうのディスクを手元のように使う仕組み	みんなで使う職員室のたな
EFS	AWS の NFS（みんなで使えるファイル置き場）	学校全体の共用たな
NFS クライアント	NFS を使う側（各タスクのカーネル）	たなを使いに来る人
close-to-open	閉じたら書き戻し、開くときに確かめる、という約束	ノートを返すときに書き終え、借りるときに最新を確かめる
O_APPEND（追記）	ファイルの最後に書き足す開き方。NFS では同時の追記は保証されない	ノートの最後のページに書き足す
ロールオーバ	ログのファイルを時間や大きさで新しくすること	ノート交換
RollingFile（A4）	Log4j2 のロールオーバつきファイル出力	交換係つきのノート
TimeBasedTriggeringPolicy	時刻でロールオーバを決める決まり（00 分を過ぎた最初のログで）	「1 時間たったら交換」の決まり
renameEmptyFiles	空のファイルを改名するか（既定は改名せず削除）	白紙のノートはしまわずに捨てる決まり
ignoreExceptions	アペンダの失敗をアプリに伝えないか（既定は伝えない）	ノート係の「書く人を困らせない」約束
StatusLogger／DefaultErrorHandler	Log4j2 自身のエラーを標準エラーに出す仕組み。同じエラーは 5 分に 1 回に間引く	職員室のメモ帳。同じことは何度も書かない
Failover アペンダ	主のアペンダが失敗したら代わりのアペンダに回す	ノートに書けなければ連絡帳に書く
CURRENT／ARCHIVE／STALE	診断ツールの判定。正しく書ける／前の時間帯のファイルに書く／Stale	机のノート／しまったノート／捨てられたノート

ECS タスク	コンテナのまとまり。Fargate ではタスクごとに別の VM	1 つの班
ECS Exec	動いているコンテナの中でコマンドを実行する仕組み	班の机をのぞきに行く
ELB／ALB のヘルスチェック	ロードバランサが一定間隔でサーバの健康を確かめる	先生の見回り
Spring MVC の Controller	HTTP のリクエストを受けて結果を返すクラス	受付係
@ControllerAdvice／@ExceptionHandler	例外をまとめて HTTP の状態に変える仕組み	困りごとの受付窓口
HTTP 200／500／503	成功／サーバの中の失敗／いまは受けられない	元気です／具合が悪い／今は休み中
fail-open	全員が不健康なら、全員に振り分ける ALB の動き	全員が休みなら、仕方なく全員に回す
CloudWatch Logs	AWS のログの保管・検索サービス	学校の記録センター
CloudTrail	AWS の操作の記録	職員室の出入り記録

17　参考資料（2026年10月10日に確認）

本文の根拠にした一次情報。Log4j2 の行番号は 2.25.3 のソース（Maven Central の sources.jar／GitHub の rel/2.25.3）。

資料	URL	確かめたこと
Log4j2 FileRenameAction（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-core/src/main/java/org/apache/logging/log4j/core/appender/rolling/action/FileRenameAction.java	空なら削除（111・194 行）、Files.move の ATOMIC_MOVE＋REPLACE_EXISTING（209 行）
Log4j2 RollingFileManager（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-core/src/main/java/org/apache/logging/log4j/core/appender/rolling/RollingFileManager.java	rollover の順番（閉じる→改名→作成）、最初の交換時刻の基準（913～932 行）
Log4j2 TimeBasedTriggeringPolicy（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-core/src/main/java/org/apache/logging/log4j/core/appender/rolling/TimeBasedTriggeringPolicy.java	00 分を過ぎた最初のイベントで交換（130～139 行）
Log4j2 DefaultErrorHandler（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-core/src/main/java/org/apache/logging/log4j/core/appender/DefaultErrorHandler.java	最初の数件のあとは 5 分に 1 件に間引く（31・99～113 行）
Log4j2 AppenderControl（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-core/src/main/java/org/apache/logging/log4j/core/config/AppenderControl.java	ignoreExceptions=true なら投げ直さない（158～173 行）
Log4j2 RollingFileAppender（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-core/src/main/java/org/apache/logging/log4j/core/appender/RollingFileAppender.java	locking が RollingFileManager に渡されない（79・144～159 行）
Log4j2 LoggerContext（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-core/src/main/java/org/apache/logging/log4j/core/LoggerContext.java	${hostName} を用意する（691 行）
log4j-api AbstractLogger（2.25.3）	https://github.com/apache/logging-log4j2/blob/rel/2.25.3/log4j-api/src/main/java/org/apache/logging/log4j/spi/AbstractLogger.java	ログ呼び出しの中の例外を抑え、LoggingException だけ投げ直す（2962～2988 行・LOG4J2-1990）
Log4j2 マニュアル：Rolling file appenders	https://logging.apache.org/log4j/2.x/manual/appenders/rolling-file.html	RollingFileManager の共有は同じ JVM の中だけ、locking の説明
Log4j2 マニュアル：File appenders	https://logging.apache.org/log4j/2.x/manual/appenders/file.html	append の説明（ローカルディスクの前提）
Log4j2 マニュアル：Delegating appenders（Failover）	https://logging.apache.org/log4j/2.x/manual/appenders/delegating.html	主のアペンダに ignoreExceptions=false が要る、retryIntervalSeconds の既定 60

Log4j リリースノート	https://logging.apache.org/log4j/2.x/release-notes.html	2.25.3〜2.26.1 の公開日と変更
Apache Logging Services セキュリティ	https://logging.apache.org/security.html	CVE-2025-68161・CVE-2026-34477／34478／34480・CVE-2026-49844
Maven Central（log4j-core の版の一覧）	https://repo1.maven.org/maven2/org/apache/logging/log4j/log4j-core/maven-metadata.xml	最新の版（2.26.1・2.25.5）
AWS：EFS の機能（一貫性・ロック）	https://docs.aws.amazon.com/efs/latest/ug/features.html	close-to-open、追記は read-after-write の保証外、任意ロックのみ
AWS：EFS マウントのトラブルシュート	https://docs.aws.amazon.com/efs/latest/ug/troubleshooting-efs-mounting.html	同じ IP で FS を作り直すと bad file handle（アンマウント・再マウントで直る）
AWS：EFS の CloudWatch メトリクス	https://docs.aws.amazon.com/efs/latest/ug/efs-metrics.html	ClientConnections の意味
AWS：ECS で EFS ボリュームを使う	https://docs.aws.amazon.com/AmazonECS/latest/developerguide/efs-volumes.html	Fargate は PV 1.4.0 以降、監督コンテナがマウントを管理
AWS：ECS Exec	https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-exec.html	動いているタスクでは後から有効にできない、読み取り専用ルートでは使えない
AWS：Fargate PV 1.4.0 の発表（2020-04-08）	https://aws.amazon.com/about-aws/whats-new/2020/04/aws-fargate-launches-platform-version-14/	Fargate で EFS が使えるようになった時期
AWS：タスクメタデータ v4（Fargate）	https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-metadata-endpoint-v4-fargate.html	起動スクリプトでタスク ID を求める
AWS：ALB のヘルスチェック	https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html	既定 30 秒・5 秒・5 回・2 回・200、全部不健康なら fail-open
AWS：ECS のコンテナヘルスチェック	https://docs.aws.amazon.com/AmazonECS/latest/developerguide/healthcheck.html	必須コンテナが UNHEALTHY ならタスクが UNHEALTHY
LKML：EFS 側の技術者の修正案（2020-11-18）	https://lkml.iu.edu/hypermail/linux/kernel/2011.2/05183.html	別のクライアントが開いているファイルを消すと ESTALE、nfsd では再現しない
LKML：NFS クライアントのメンテナの返信	https://lkml.iu.edu/2011.2/05208.html	NFS4ERR_STALE は「その実体へのアクセスは取り消された」という意味
Linux NFS FAQ（A8・A9・A10）	https://nfs.sourceforge.net/	close-to-open（2.4.20）、追記は原子的でない、ESTALE の意味
Linux man-pages：open(2)	https://man7.org/linux/man-pages/man2/open.2.html	O_APPEND は NFS で複数の書き手だと壊れることがある
RFC 1094（NFSv2）	https://www.rfc-editor.org/rfc/rfc1094	ファイルハンドルとステートレスの設計
RFC 1813（NFSv3）	https://www.rfc-editor.org/rfc/rfc1813	NFS3ERR_STALE
RFC 7530（NFSv4.0）	https://www.rfc-editor.org/rfc/rfc7530	OPEN の状態を持つ NFS
RFC 8881（NFSv4.1）	https://www.rfc-editor.org/rfc/rfc8881	OPEN4_RESULT_PRESERVE_UNLINKED（残すかはサーバ次第）

The Twelve-Factor App : Logs	https://12factor.net/logs	ログはイベントの流れ（標準出力へ）
Red Hat Developer : JBoss EAP 8.1	https://developers.redhat.com/blog/2025/09/02/jboss-eap-81-modernizing-enterprise-java-applications	EAP 8.1 の公開（2025 年 9 月）
Red Hat : RHEL 9.8 リリースノート	https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/9.8_release_notes/overview	RHEL 9.8（2026 年 5 月）
log4j の歴史（Apache の history）	https://svn.apache.org/repos/asf/logging/log4j/tags/v_1_1b2/docs/history.html	1996 年の SEMPER、IBM、2001 年に Apache へ
前回の資料（このフォルダの隣）	C:¥Users¥taka_¥Claude¥Log_Stale_Handle	2 タスク・gz 圧縮の構成での検証（v1）。今回は 6 タスク・本番の設定で作り直した